

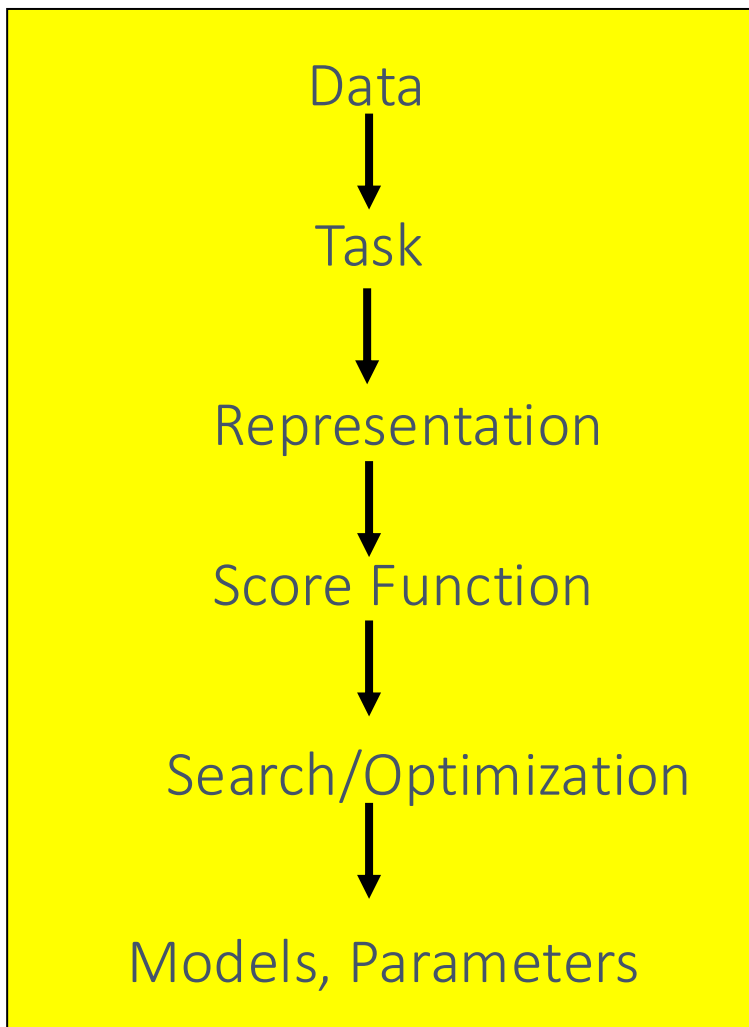
UVA CS 4774: Machine Learning

Lecture 3: Linear Regression Basics

Dr. Yanjun Qi

University of Virginia
Department of Computer Science

Machine Learning in a Nutshell



ML grew out of
work in AI

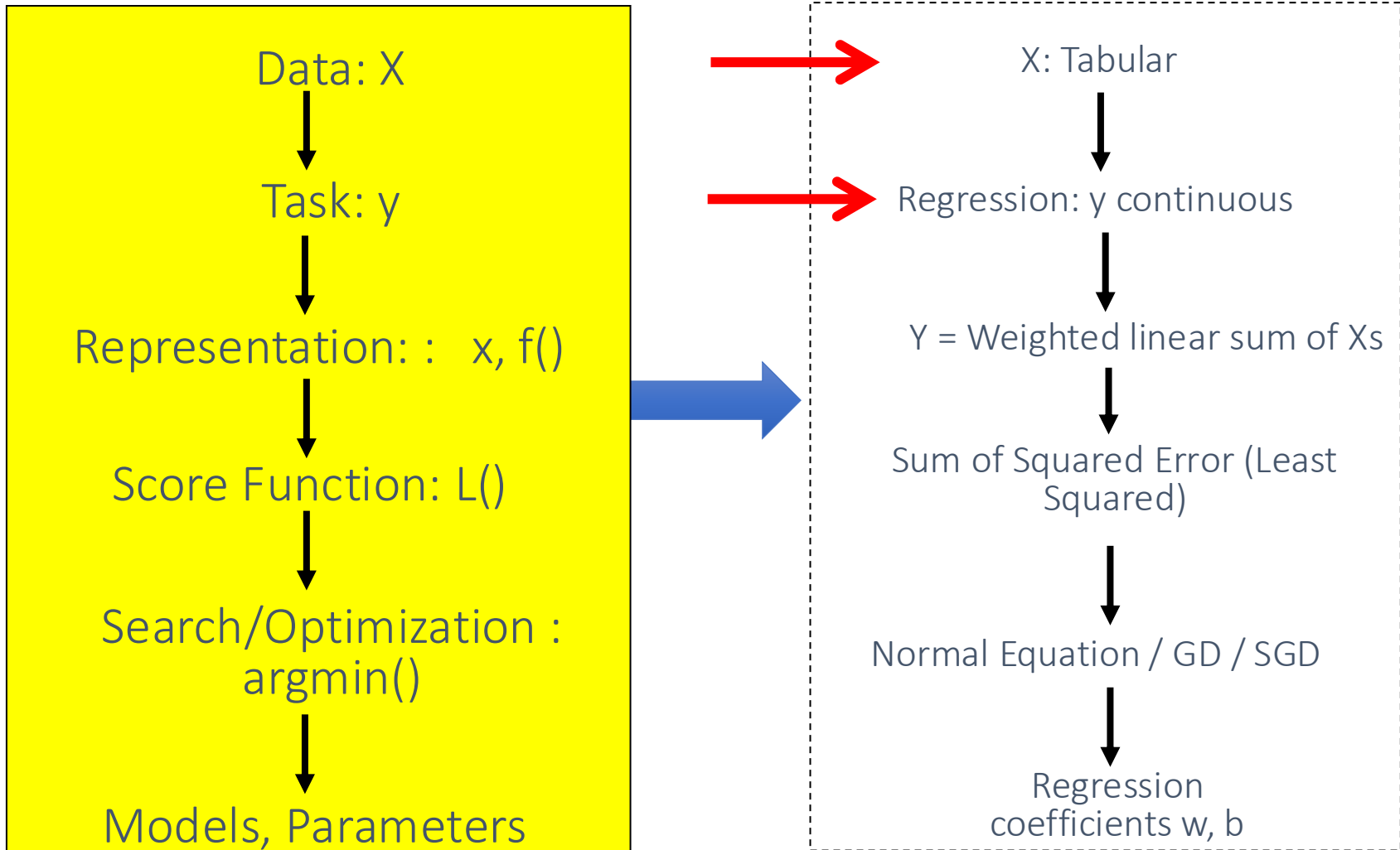
Optimize a
performance criterion
using example data or
past experience,

Aiming to generalize to
unseen data

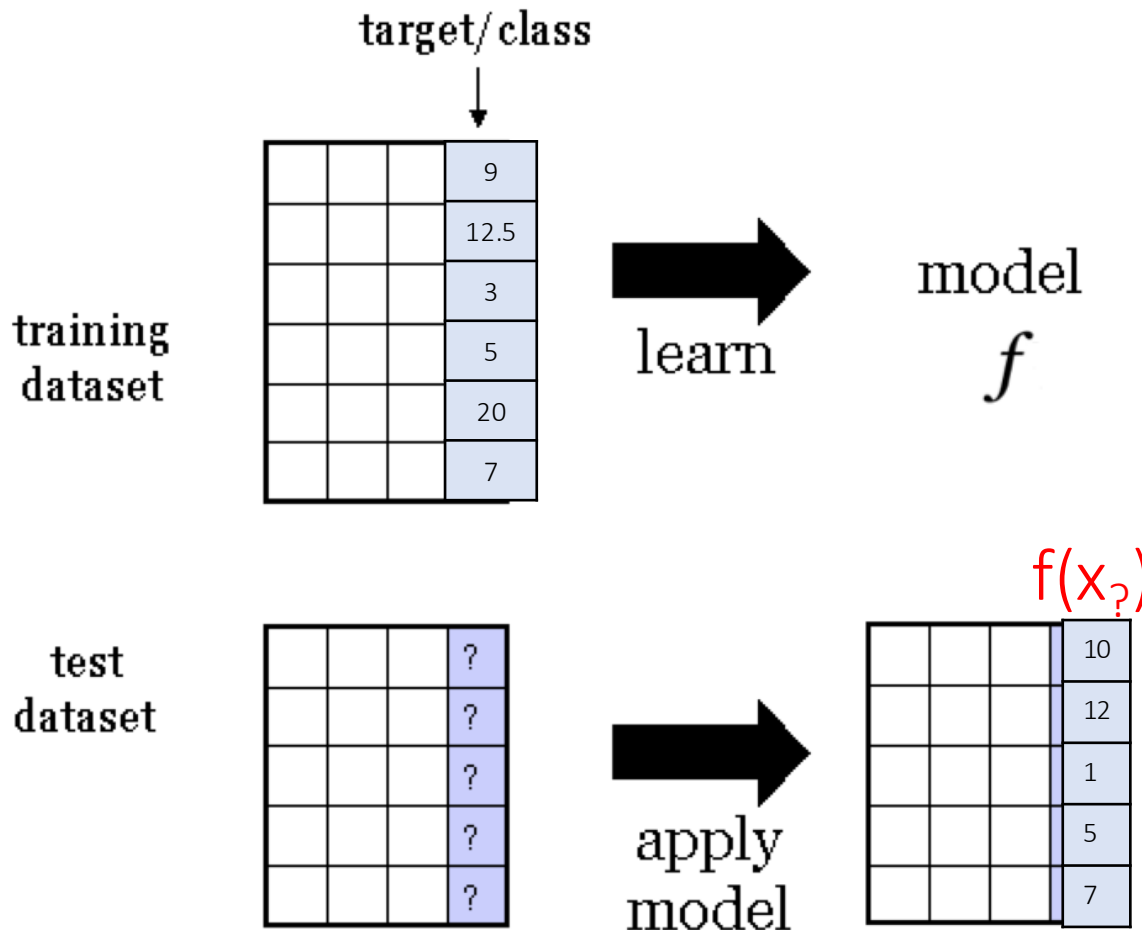
Rough Sectioning of this Course

- 
- 1. Basic Supervised Regression + on Tabular Data
 - 2. Basic Deep Learning + on 2D Imaging Data
 - 3. Generative and Deep + on 1D Sequence Text Data
 - 4. Advanced Supervised learning + on Tabular Data
 - 5. Advanced topics like Fine Tuning, Agent, RAG

Today: Multivariate Linear Regression in a Nutshell



SUPERVISED Regression



Training dataset consists of **input-output** pairs

- Target Y: **continuous** target variable

training dataset $\rightarrow$

$$\mathbf{X}_{train} = \begin{bmatrix} - & - & \mathbf{x}_1^T & - & - \\ - & - & \mathbf{x}_2^T & - & - \\ \vdots & & \vdots & & \vdots \\ - & - & \mathbf{x}_n^T & - & - \end{bmatrix}$$

$$\bar{\mathbf{y}}_{train} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

$n \times p$

test dataset $\rightarrow$

$$\mathbf{X}_{test} = \begin{bmatrix} - & - & \mathbf{x}_{n+1}^T & - & - \\ - & - & \mathbf{x}_{n+2}^T & - & - \\ \vdots & & \vdots & & \vdots \\ - & - & \mathbf{x}_{n+m}^T & - & - \end{bmatrix}$$

$$\bar{\mathbf{y}}_{test} = \begin{bmatrix} y_{n+1} \\ y_{n+2} \\ \vdots \\ y_{n+m} \end{bmatrix}$$

$m \times p$

$f(\mathbf{X}_{test})$

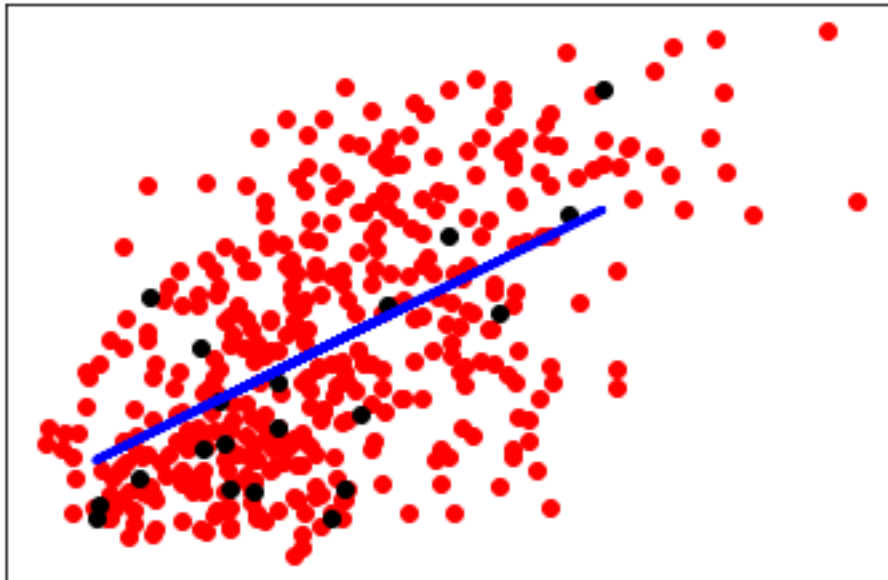
$m \times 1$

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

<https://colab.research.google.com/drive/1mI-PFB2J1UVlp6JjLhQKEPirPZlQnDH7?usp=sharing>

I will code-run through: Scikit-Learn Linear Regression Example

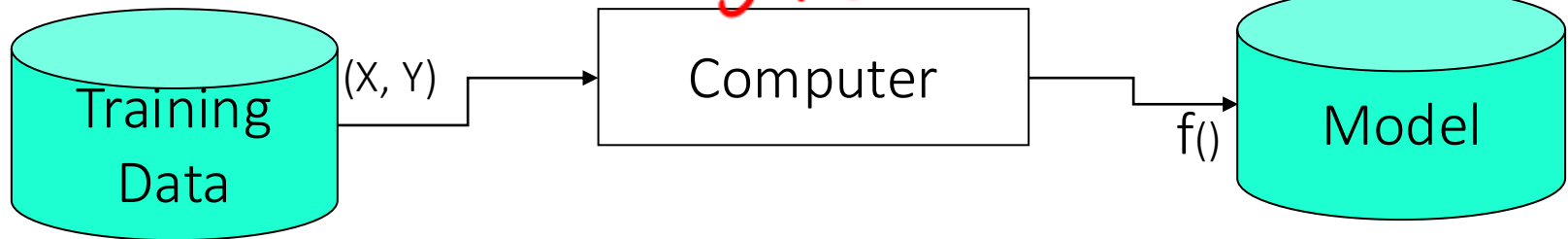
Adapted from: https://scikit-learn.org/stable/auto_examples/linear_model/plot_ols.html



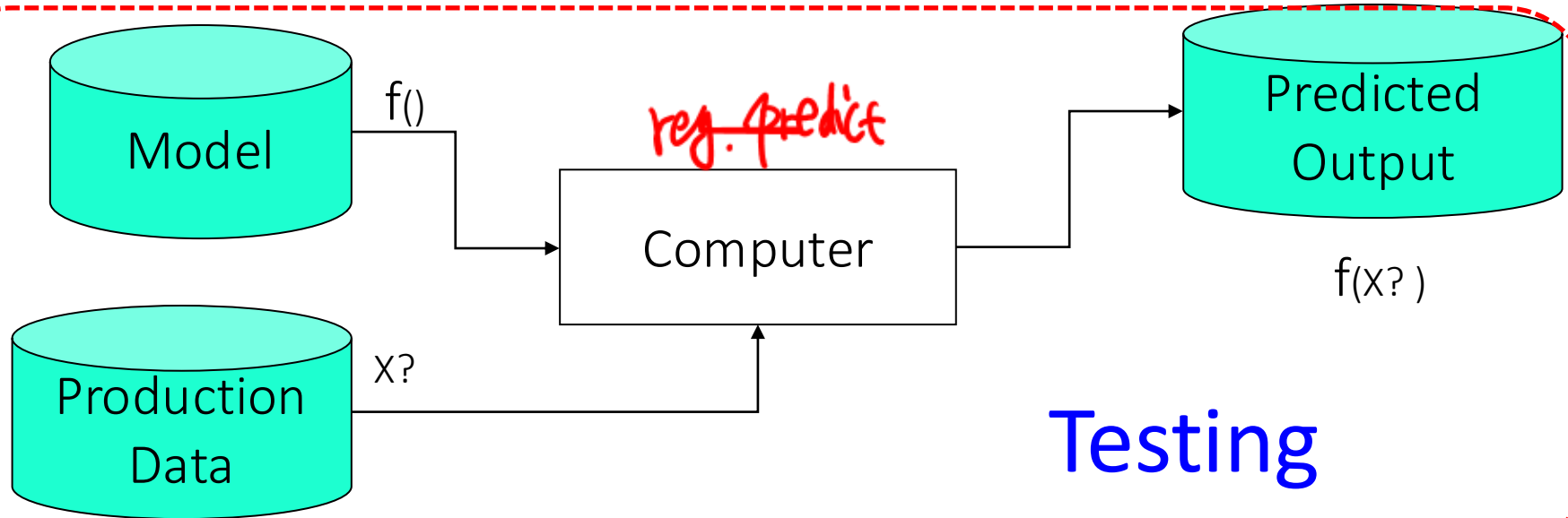
Two Modes of Machine Learning

Consists of **input-output** pairs

Training



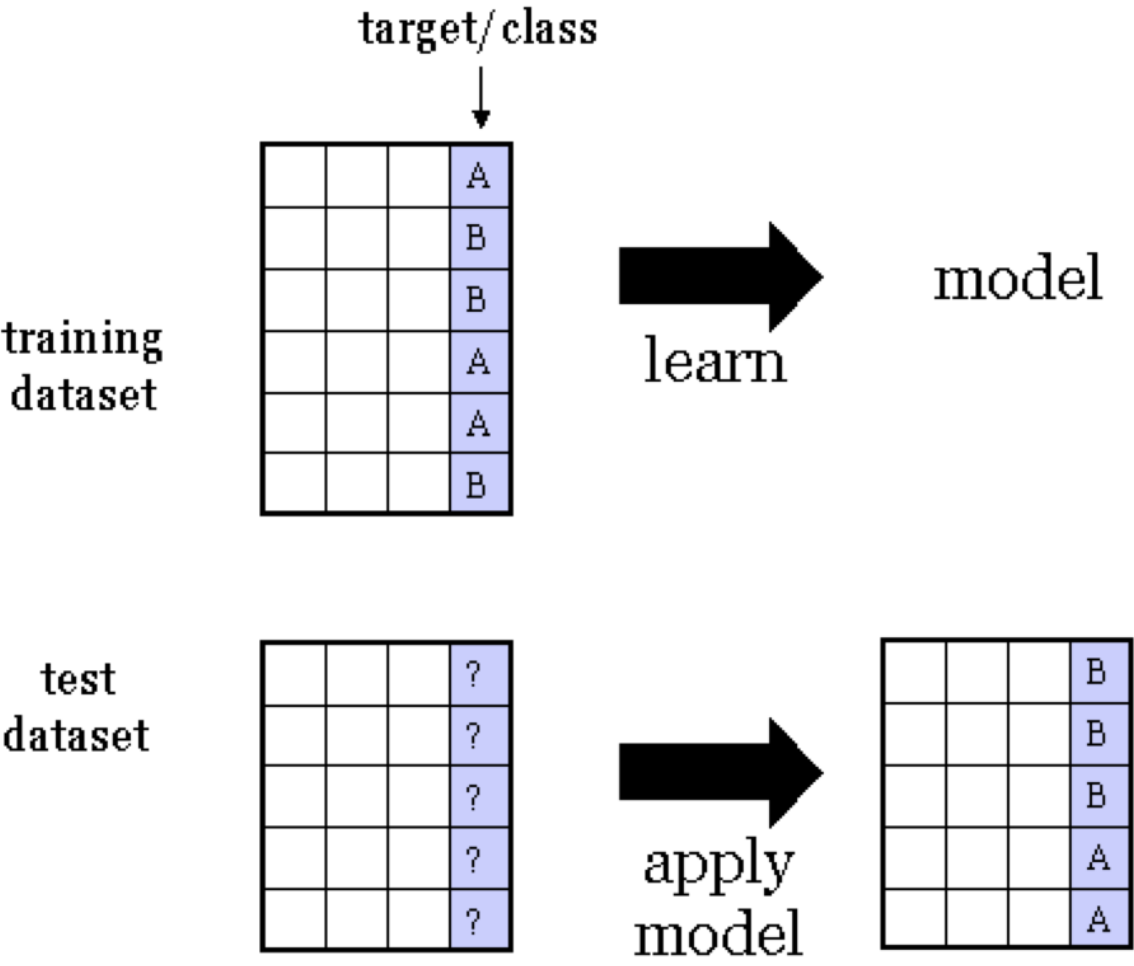
Testing



<https://scikit-learn.org/stable/tutorial/basic/tutorial.html>

https://scikit-learn.org/stable/auto_examples/classification/plot_digits_classification.html#sphx-gl-auto-examples-classification-plot-digits-classification-py

Another exemplar notebook : Recognizing hand-written digits with L2.ipynb
Adapted from: ScikitLearn Tutorial plot_digits_classification.ipynb

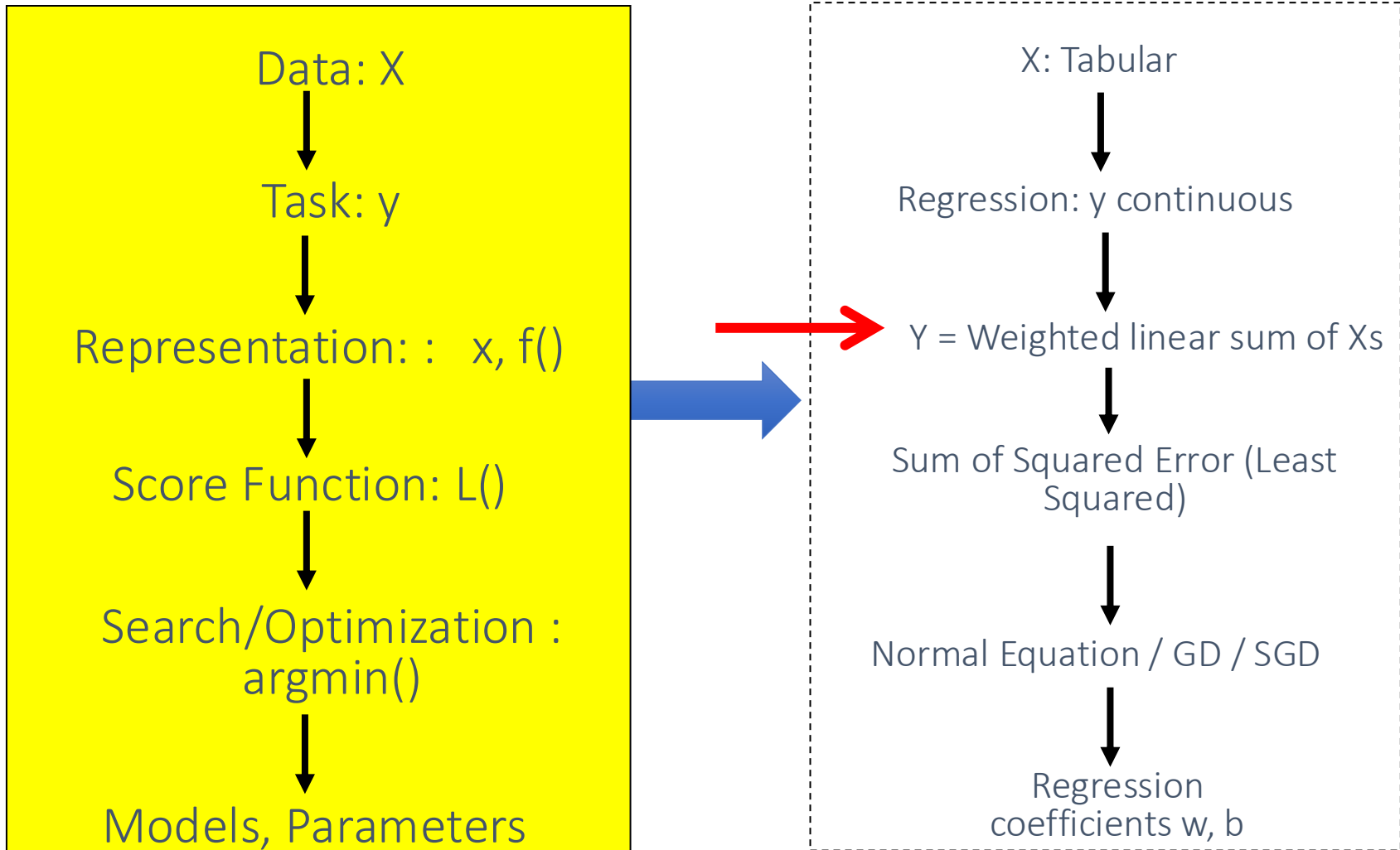


Training dataset consists of **input-output** pairs

Thank You



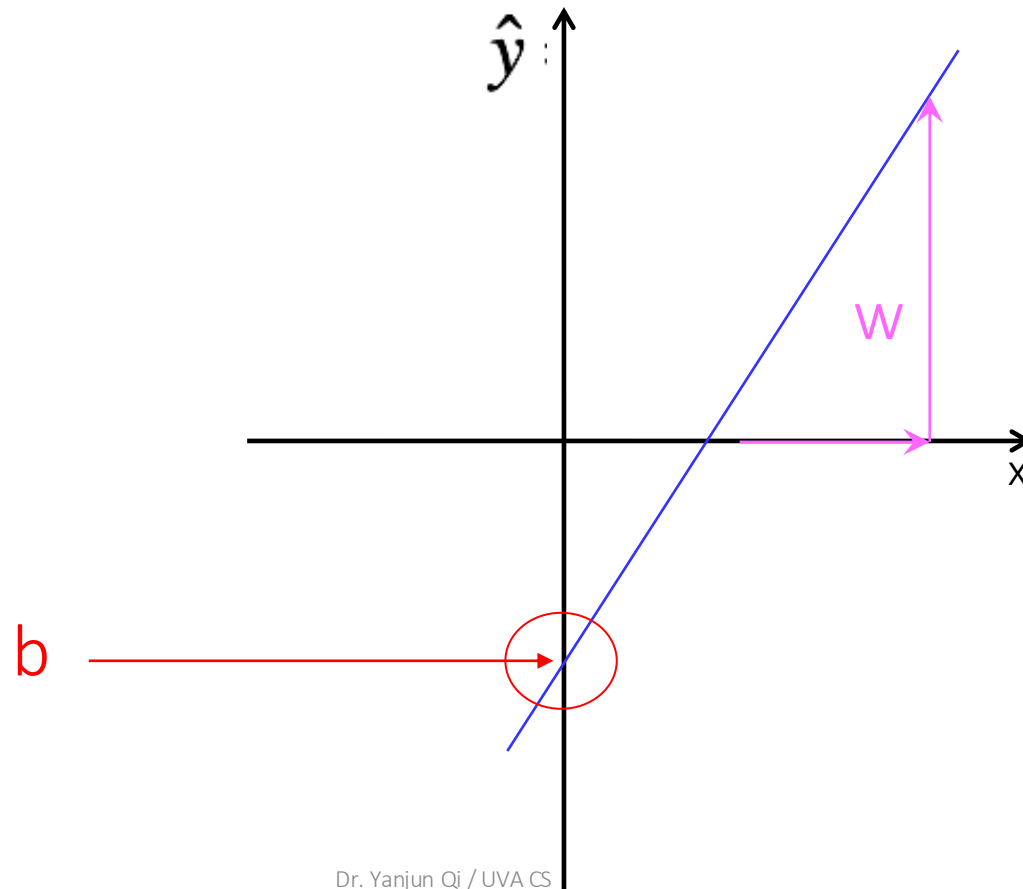
Today: Multivariate Linear Regression in a Nutshell



Review: $f(x)$ is Linear when X is single variable

- $f(x)=wx+b$?

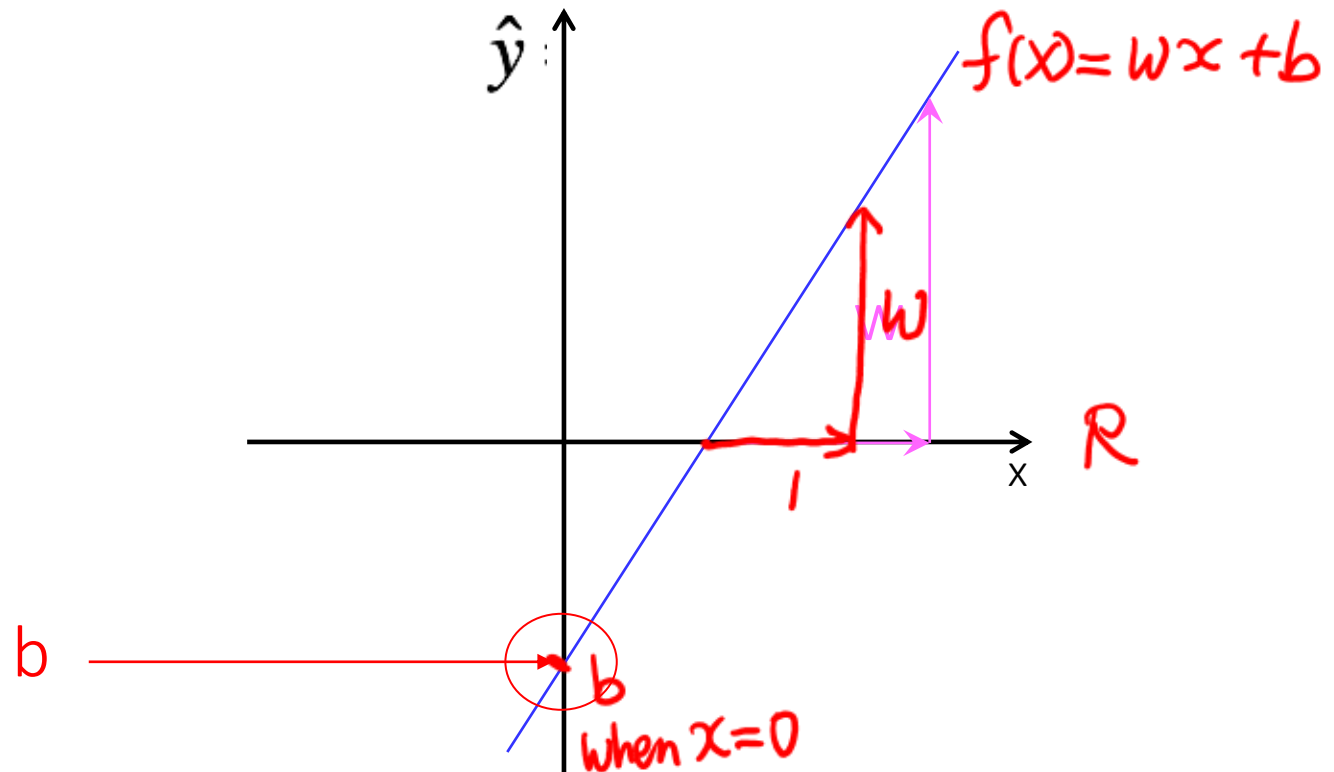
A **slope** of 2 (i.e. $w=2$) means that every 1-unit change in X yields a 2-unit change in Y .



Review: $f(x)$ is Linear when X is single variable

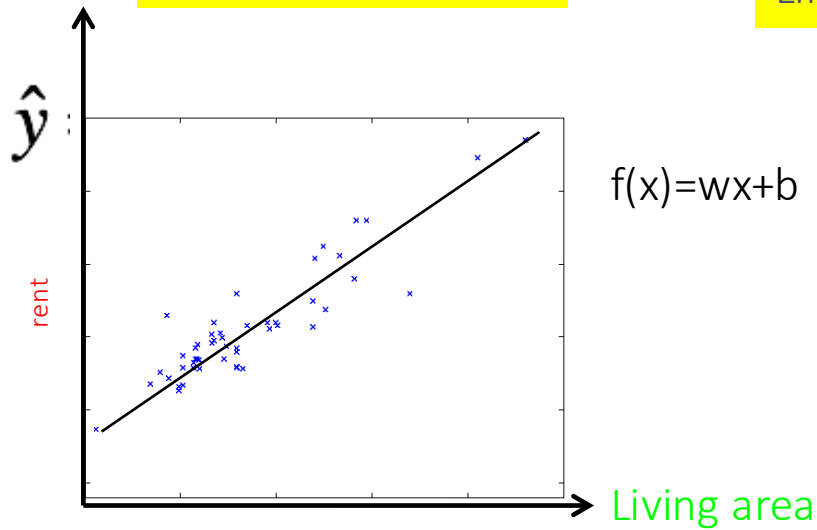
- $f(x) = wx + b$?

A slope of 2 (i.e. $w=2$) means that every 1-unit change in X yields a 2-unit change in Y .



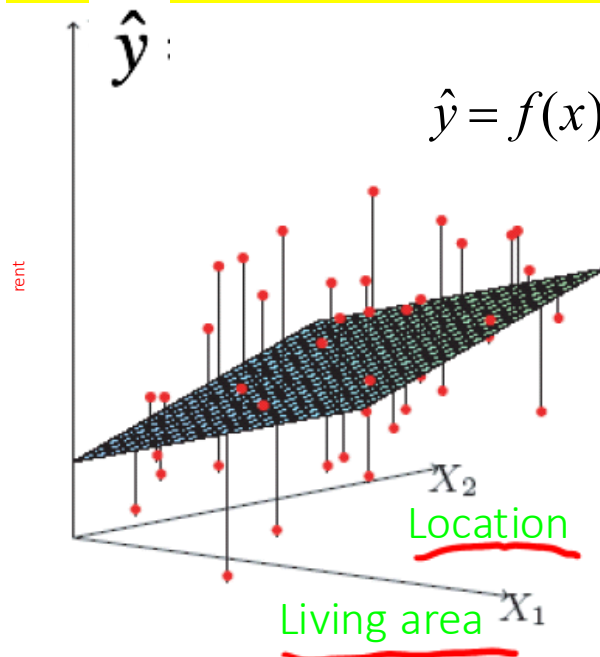
e.g., Living area as X

Linear Regression: Y as Weighted linear sum of Xs



1D case ($\mathcal{X} = \mathbb{R}$): a line

e.g., (Living area, Location) as X



weighted sum of x_i

$\mathcal{X} = \mathbb{R}^2$: a plane

Linear Supervised Regression

$$f: X \longrightarrow Y$$

Linear Regression Models

Linear Regression: Y as Weighted linear sum of Xs

$$\hat{y} = f(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

=> Features x_i :

e.g., Living area, distance to campus, # bedroom ...

=> Target y :

e.g., Rent (a continuous variable)

Linear Supervised Regression

$$f: X \longrightarrow Y$$

Linear Regression Models

Linear Regression: Y as Weighted linear sum of Xs

$$\hat{y} = f(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

=> Features x_i :

e.g., Living area, distance to campus, # bedroom ...

=> Target y :

e.g., Rent (a continuous variable)

Linear Supervised Regression

$$f: X \longrightarrow Y$$

$$f(x) = \theta_0 + \theta_1 x_1 + \dots + \theta_p x_p$$

Linear Regression Models

Linear Regression: Y as Weighted linear sum of Xs

$$\hat{y} = f(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

=> Features x_i : $\vec{x} \in \mathbb{R}^p$:

e.g., Living area, distance to campus, # bedroom ...

=> Target y :

e.g., Rent (a continuous variable)

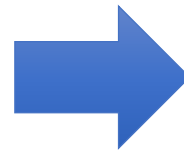
A Concise Notation: via Vector/Matrix Product

- Represent each data sample $\mathbf{x}$ as a column vector, plus a pseudo feature
 - We add a pseudo "feature" $x_0=1$ (this is the intercept term), and re-define the feature vector to be:

$$\vec{x} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_p \end{bmatrix}$$

- The parameter vector θ is also a column vector

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_p \end{bmatrix}$$



$$\hat{y} = f(\mathbf{x})$$

$$\hat{y} = \mathbf{x}^T \theta = \theta^T \mathbf{x}$$

$$\hat{y} = f(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

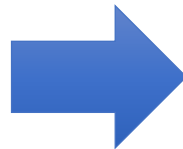
A Concise Notation: via Vector/Matrix Product

- Represent **each data sample** $\mathbf{x}$ as a column vector, plus a pseudo feature
 - We add a pseudo "feature" $x_0=1$ (this is the **intercept** term), and **re-define** the feature vector to be:

$$\mathbf{x}^T = [(\underline{x_0=1}), x_1, x_2, \dots, x_p]$$

- The parameter vector θ is also a column vector

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_p \end{bmatrix}$$



$$\hat{y} = f(\mathbf{x})$$

$$\hat{y} = \mathbf{x}^T \theta = \theta^T \mathbf{x}$$

$$= \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_p x_p$$

$$\hat{y} = f(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

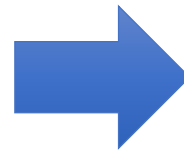
A Concise Notation: via Vector/Matrix Product

- Represent **each data sample** $\mathbf{x}$ as a column vector, plus a pseudo feature
 - We add a pseudo "feature" $x_0=1$ (this is the **intercept** term), and **re-define** the feature vector to be:

- The parameter vector θ is also a **column vector**

$$\vec{x} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_p \end{bmatrix}$$

$$\vec{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_p \end{bmatrix} \quad (p+1) \times 1$$



$$\hat{y} = f(\mathbf{x})$$

$$\hat{y} = \vec{x}^T \vec{\theta} = \theta^T \mathbf{x}$$

$$\hat{y} = f(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 \quad \Rightarrow \quad [1 \ x_1 \ x_2 \dots x_p] \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_p \end{bmatrix}$$

Review:

- Dot (or Inner) Product of two vectors $\langle \mathbf{x}, \mathbf{a} \rangle$

is the sum of products of elements in similar positions for the two vectors

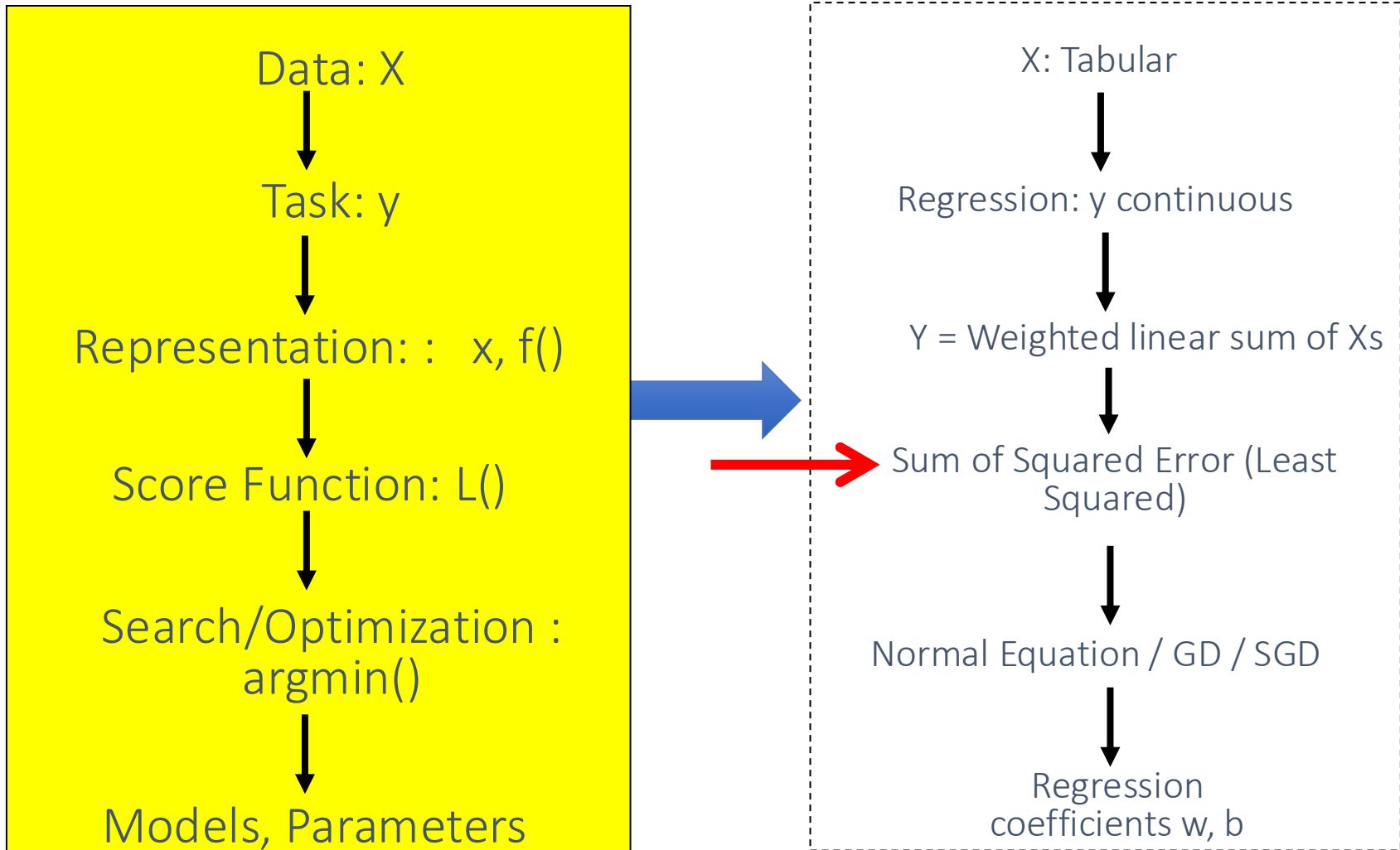
$$\langle \mathbf{x}, \boldsymbol{\Theta} \rangle = \langle \boldsymbol{\Theta}, \mathbf{x} \rangle \quad \mathbf{x}^T \boldsymbol{\theta} = \boldsymbol{\theta}^T \mathbf{x}$$

Linear Regression: $\hat{y}$ as Weighted linear sum of $\mathbf{x}$ s

$$\hat{y} = f(\mathbf{x})$$

$$\hat{y} = \mathbf{x}^T \boldsymbol{\theta} = \boldsymbol{\theta}^T \mathbf{x}$$

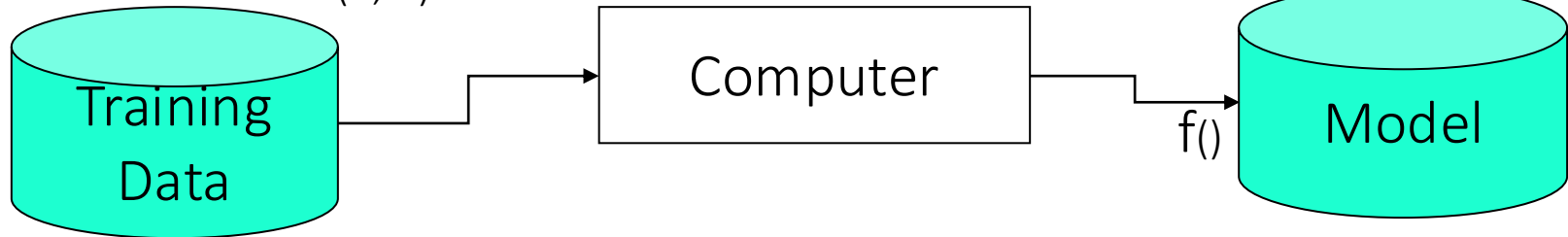
Today: Multivariate Linear Regression in a Nutshell



Training Modes of Machine Learning

Consists of **input-output** pairs

(X, Y)



```
# Split the data into training/testing sets
diabetes_X_train = diabetes_X[:-20]
diabetes_X_test = diabetes_X[-20:]

# Split the targets into training/testing sets
diabetes_y_train = diabetes_y[:-20]
diabetes_y_test = diabetes_y[-20:]

# Create linear regression object
regr = linear_model.LinearRegression()

# Train the model using the training sets
regr.fit(diabetes_X_train, diabetes_y_train)
```


Basic Concepts

- Training (i.e. learning parameters w, b)
 - Training set includes
 - available examples $x_1, \dots, x_n$
 - available corresponding labels $y_1, \dots, y_n$
- Find (w, b) by minimizing loss / Cost function $L()$
 - (i.e. difference between y and $f(x)$ on available examples in training set)

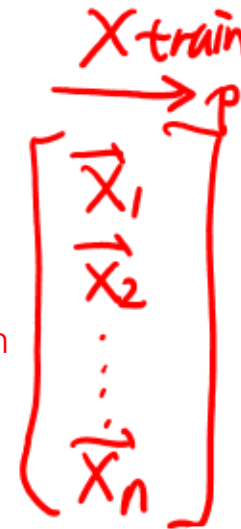
$$(W, b) = \underset{w, b}{\operatorname{argmin}} \sum_{i=1}^n \ell(f(x_i), y_i)$$

Basic Concepts

- Training (i.e. learning parameters w, b)

- Training set includes

- available examples $x_1, \dots, x_n$
- available corresponding labels $y_1, \dots, y_n$



- Find (w, b) by minimizing loss / Cost function $L()$
 - (i.e. difference between y and $f(x)$ on available examples in training set)

$$(W, b) = \underset{w, b}{\operatorname{argmin}} \sum_{i=1}^n \ell(\underline{f(x_i)}, \underline{y_i})$$

Loss/Cost function for Regression $L()$

SSE: Sum of squared error

- Our goal is to search for the optimal θ that minimize the following lost :

$$J(\theta) = \frac{1}{2} \sum_{i=1}^n (f(\mathbf{x}_i) - y_i)^2$$

Loss/Cost function for Regression $L()$

SSE: Sum of squared error

- Our goal is to search for the optimal θ that minimize the following lost :

$$J(\theta) = \frac{1}{2} \sum_{i=1}^n (f(\mathbf{x}_i) - y_i)^2$$

$$\hat{y} = f(\mathbf{x})$$

$$\mathbf{x}^T \theta = \theta^T \mathbf{x}$$

Loss/Cost function for Regression $L()$

SSE: Sum of squared error

- Our goal is to search for the optimal θ that minimize the following lost :

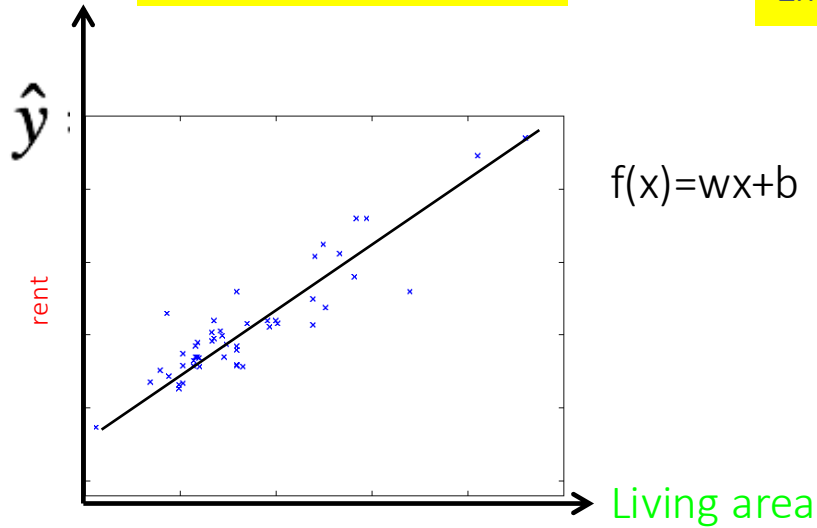
training: $J(\theta) = \frac{1}{2} \sum_{i=1}^n (f(\mathbf{x}_i) - y_i)^2 \Rightarrow \theta^* = \underset{\theta}{\operatorname{argmin}} J(\theta)$

testing: $\hat{y} = f(\mathbf{x})$

$$\mathbf{x}^T \theta^* = \theta^{*T} \mathbf{x}$$

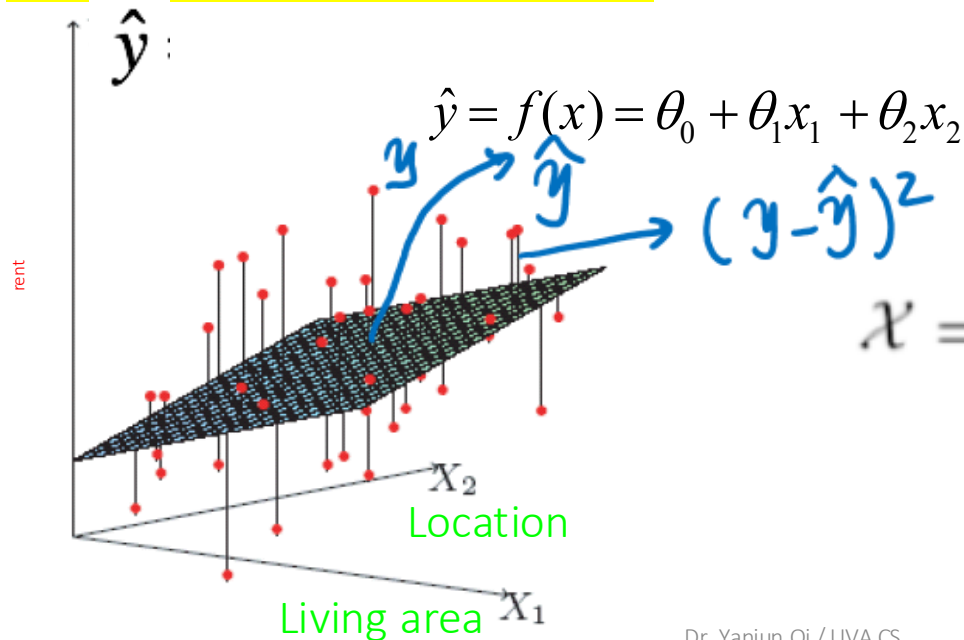
e.g., Living area as X

Linear Regression: Y as Weighted linear sum of Xs



1D case ($\mathcal{X} = \mathbb{R}$): a line

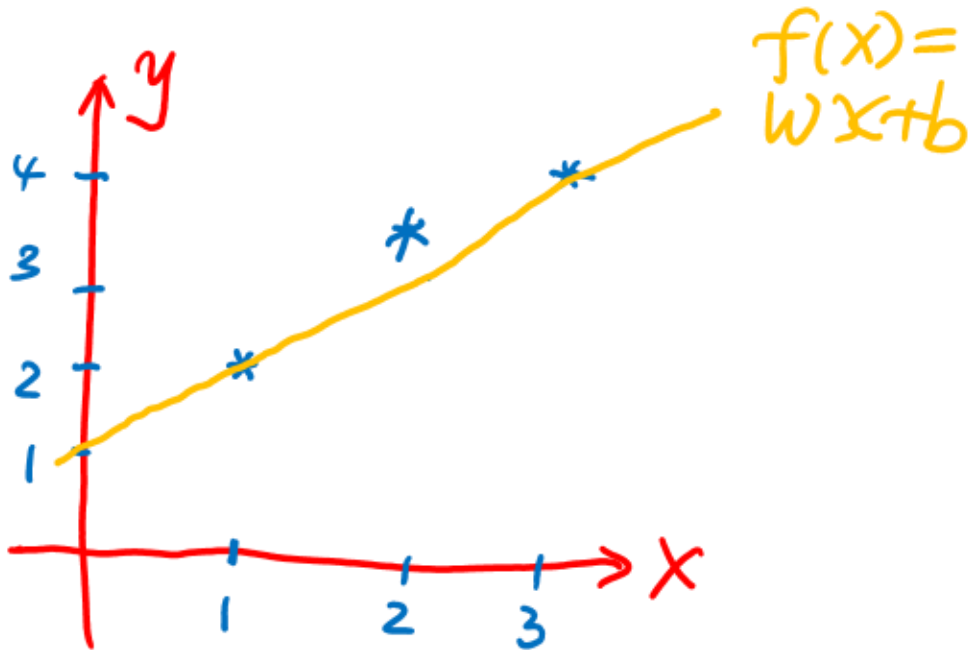
e.g., (Living area, Location) as X



$\mathcal{X} = \mathbb{R}^2$: a plane

One concrete example when $p=1$

	x	y
S_1	1	2
S_2	2	3.3
S_3	3	4



$\Downarrow$

$$f(x) = wx + b$$

$\Downarrow$

$$(w^*, b^*) = \underset{w, b}{\operatorname{argmin}}$$

$$J(w, b) = \sum_{i=1}^3 (wx_i + b - y_i)^2$$

One concrete example

$n=3$ {

x_i	y_i	$f(x_i) = \hat{y}_i = wx + b$	$(wx_i + b - y_i)^2$
1	2	$w + b$	$(w + b - 2)^2$
2	3	$2w + b$	$(2w + b - 3)^2$
3	4	$3w + b$	$(3w + b - 4)^2$

$$J(\overline{w}, b) = \frac{1}{2} \left\{ (w + b - 2)^2 + (2w + b - 3)^2 + (3w + b - 4)^2 \right\}$$

$$\Rightarrow ?? \quad \underset{w, b}{\operatorname{argmin}} J(w, b)$$

Now the loss function: via Concise Notation

- Using matrix form, we get the following general representation of the linear regression function:

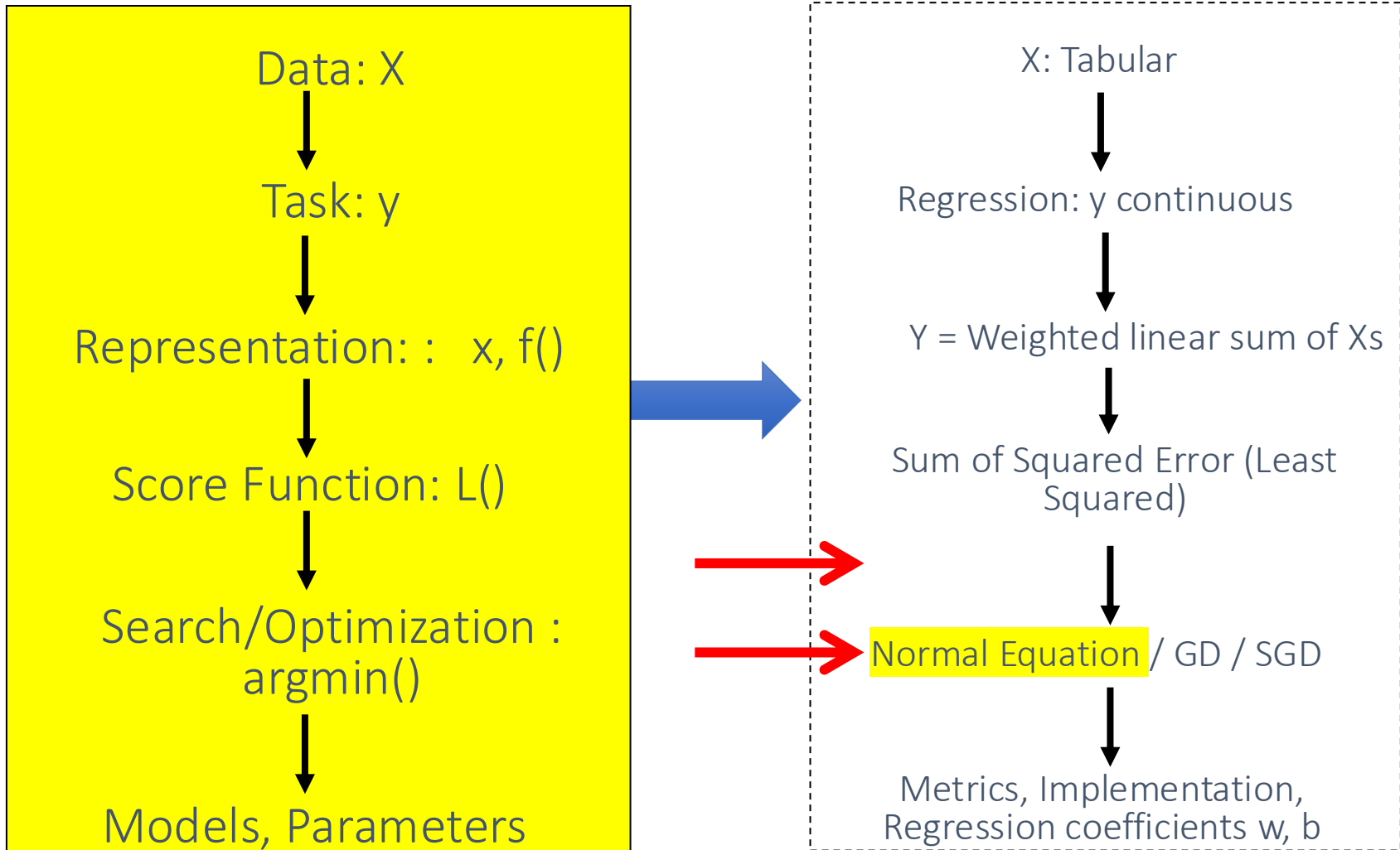
$$\mathbf{X} = \begin{bmatrix} -- & \mathbf{x}_1^T & -- \\ -- & \mathbf{x}_2^T & -- \\ \vdots & \vdots & \vdots \\ -- & \mathbf{x}_n^T & -- \end{bmatrix}$$

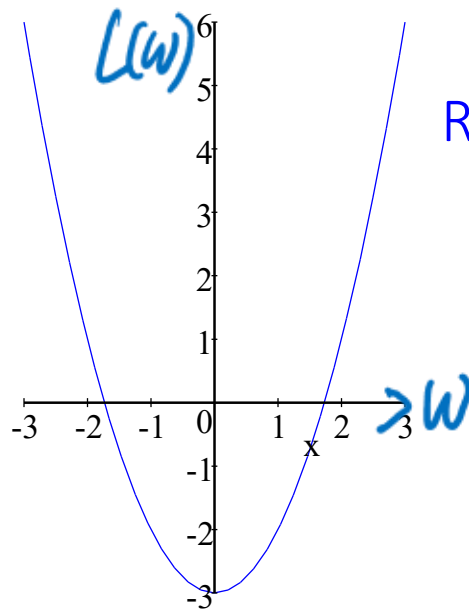
$$\begin{aligned} J(\theta) &= \frac{1}{2} \sum_{i=1}^n (\mathbf{x}_i^T \theta - y_i)^2 \\ &= \frac{1}{2} (X\theta - \bar{y})^T (X\theta - \bar{y}) \\ &= \frac{1}{2} (\theta^T X^T X \theta - \theta^T X^T \bar{y} - \bar{y}^T X \theta + \bar{y}^T \bar{y}) \end{aligned}$$

Thank You



Today: Multivariate Linear Regression in a Nutshell





Review (IV): Derivative of a Quadratic Function

$$\underline{L(w)} = w^2 - 3$$

$$L'(w) = 2w = 0$$

$$\operatorname{argmin}_w L(w)$$

$$= (L'(w) = 0) \parallel w^*$$

This quadratic (convex) function is minimized @ the unique point whose derivative (slope) is zero.

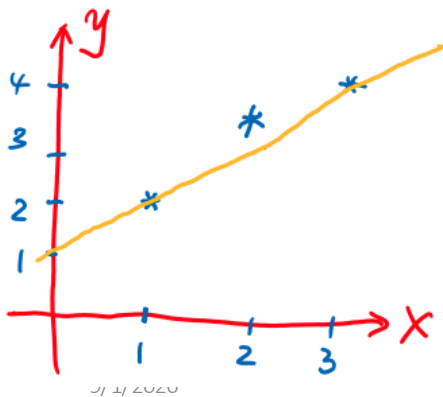
➔ When we find zeros of the derivative of this function, we also find the minima (or maxima) of that function.

$$f(x) = wx + b$$

Our concrete example

$$\Rightarrow J(w, b) = \frac{1}{2} \left((w+b-2)^2 + (2w+b-3)^2 \right)$$

$$\Rightarrow \begin{cases} \frac{\partial J(w, b)}{\partial w} = (w+b-2) + (2w+b-3) \cdot 2 = 0 \\ \frac{\partial J(w, b)}{\partial b} = w+b-2 + (2w+b-3) = 0 \end{cases}$$



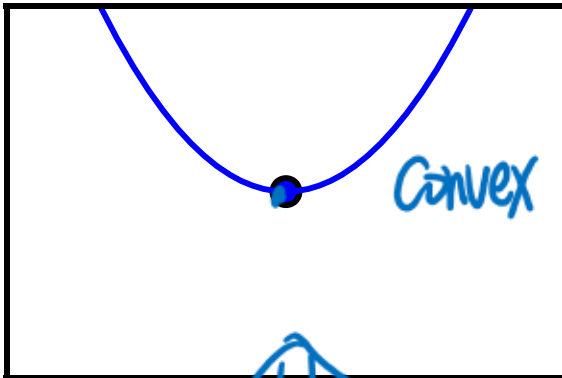
$$\begin{cases} 5w + 3b - 8 = 0 \\ 3w + 2b - 5 = 0 \end{cases} \Rightarrow \begin{cases} 10w + 6b - 16 = 0 \\ 9w + 6b - 15 = 0 \end{cases}$$

$$\Rightarrow w = 1, b = \frac{1}{2}(5 - 3w) = 1$$

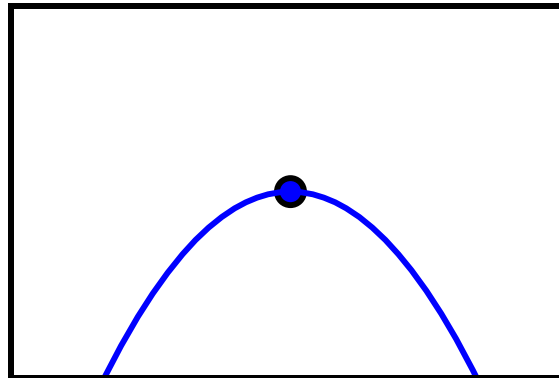
we solve the matrix equation via Gaussian Elimination

Critical Points

Minimum



Maximum



Saddle point

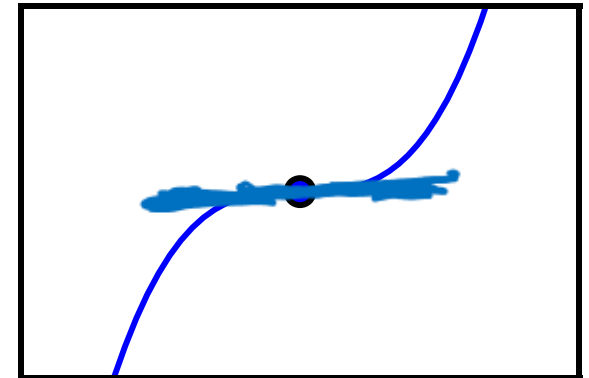


Figure 4.2

$$J(\theta) = \sum_{i=1}^n (\vec{x}_i^T \vec{\theta} - y_i)^2$$

SSE is an extremely friendly
convex loss function

Now return to matrix vector concise form:

$$\vec{X}_1 = \begin{bmatrix} 1 \\ x_{1,1} \\ x_{1,2} \\ x_{1,3} \\ \vdots \\ x_{1,p} \end{bmatrix} \quad (p+1) \times 1$$

$$\vec{X}_1^T = [1, x_{1,1}, x_{1,2}, \dots, x_{1,p}]$$

$$\mathbf{X} = \begin{bmatrix} \text{--} & \mathbf{x}_1^T & \text{--} \\ \text{--} & \mathbf{x}_2^T & \text{--} \\ \vdots & \vdots & \vdots \\ \text{--} & \mathbf{x}_n^T & \text{--} \end{bmatrix}$$

Now return to matrix vector concise form:

$$\vec{X}_1 = \begin{bmatrix} 1 \\ x_{1,1} \\ x_{1,2} \\ x_{1,3} \\ \vdots \\ x_{1,p} \end{bmatrix}_{(p+1) \times 1}$$

$$\vec{X}_1^T = [1, x_{1,1}, x_{1,2}, \dots, x_{1,p}]$$

$$\mathbf{X} = \begin{bmatrix} \text{--} & \mathbf{x}_1^T & \text{--} \\ \text{--} & \mathbf{x}_2^T & \text{--} \\ \vdots & \vdots & \vdots \\ \text{--} & \mathbf{x}_n^T & \text{--} \end{bmatrix}$$

$\downarrow$
 $n \times p$

Training Set in Matrix Form

- the whole Training set (with n samples) as matrix form :

$$\mathbf{X} = \begin{bmatrix} \text{---} & \mathbf{X}_1^T & \text{---} \\ \text{---} & \mathbf{X}_2^T & \text{---} \\ \vdots & \vdots & \vdots \\ \text{---} & \mathbf{X}_n^T & \text{---} \end{bmatrix} = \begin{bmatrix} X_{1,0} & X_{1,1} & \cdots & X_{1,p} \\ X_{2,0} & X_{2,1} & \cdots & X_{2,p} \\ \vdots & \vdots & \vdots & \vdots \\ X_{n,0} & X_{n,1} & \cdots & X_{n,p} \end{bmatrix} \quad \bar{\mathbf{y}}_{train} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

	X_1	X_2	Y
S_1			
S_2			
S_3			
S_4			
S_5			
S_6			

Cont: Loss function: via Concise Notation

- Using matrix form, we get the following general representation of the linear regression function:

$$\mathbf{X} = \begin{bmatrix} -- & \mathbf{x}_1^T & -- \\ -- & \mathbf{x}_2^T & -- \\ \vdots & \vdots & \vdots \\ -- & \mathbf{x}_n^T & -- \end{bmatrix}$$
$$\begin{aligned} J(\theta) &= \frac{1}{2} \sum_{i=1}^n (\mathbf{x}_i^T \theta - y_i)^2 \\ &= \frac{1}{2} (X\theta - \bar{y})^T (X\theta - \bar{y}) \\ &= \frac{1}{2} (\theta^T X^T X \theta - \theta^T X^T \bar{y} - \bar{y}^T X \theta + \bar{y}^T \bar{y}) \end{aligned}$$

Training Set in Matrix Form

- the whole Training set (with n samples) as matrix form :

$$\mathbf{X} \boldsymbol{\theta} = \begin{bmatrix} \text{---} & \mathbf{x}_1^T & \text{---} \\ \text{---} & \mathbf{x}_2^T & \text{---} \\ \vdots & \vdots & \vdots \\ \text{---} & \mathbf{x}_n^T & \text{---} \end{bmatrix} \boldsymbol{\theta} = \begin{bmatrix} x_{1,0} & x_{1,1} & \cdots & x_{1,p} \\ x_{2,0} & x_{2,1} & \cdots & x_{2,p} \\ \vdots & \vdots & \vdots & \vdots \\ x_{n,0} & x_{n,1} & \cdots & x_{n,p} \end{bmatrix} \bar{\mathbf{y}}_{train} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

Handwritten notes above the matrix equation:

- $\sum \theta = \begin{bmatrix} x_1^T \theta \\ \vdots \\ x_n^T \theta \end{bmatrix}$ (in red)
- $n \times (p+1)$ (in blue, under the first matrix)
- $(p+1) \times 1$ (in blue, under the second matrix)

	X ₁	X ₂	Y
s ₁			
s ₂			
s ₃			
s ₄			
s ₅			
s ₆			

Cont: Loss function: via A Concise Notation

- Using matrix form, we get the following general representation of the linear regression function:

$$\mathbf{X} = \begin{bmatrix} \text{--} & \mathbf{x}_1^T & \text{--} \\ \text{--} & \mathbf{x}_2^T & \text{--} \\ \vdots & \vdots & \vdots \\ \text{--} & \mathbf{x}_n^T & \text{--} \end{bmatrix} \quad \theta$$

$$J(\theta) = \frac{1}{2} \sum_{i=1}^n (\mathbf{x}_i^T \theta - y_i)^2$$

$$= \frac{1}{2} (\mathbf{X}\theta - \bar{\mathbf{y}})^T (\mathbf{X}\theta - \bar{\mathbf{y}})$$

$$= \frac{1}{2} (\theta^T \mathbf{X}^T \mathbf{X} \theta - \theta^T \mathbf{X}^T \bar{\mathbf{y}} - \bar{\mathbf{y}}^T \mathbf{X} \theta + \bar{\mathbf{y}}^T \bar{\mathbf{y}})$$

$$\rightarrow \mathbf{a} = (\mathbf{X}\theta - \bar{\mathbf{y}}) = \begin{bmatrix} \mathbf{x}_1^T \theta - y_1 \\ \mathbf{x}_2^T \theta - y_2 \\ \vdots \\ \mathbf{x}_n^T \theta - y_n \end{bmatrix}$$

$\underbrace{n \times p \quad p \times 1}_{n \times 1} \quad \underbrace{n \times 1}_{n \times 1}$

$$\vec{a}^T \vec{a} = \sum_{i=1}^n a_i^2$$

Find best θ via Solving $\nabla_{\theta} J(\theta) = 0$

- Write the cost function in matrix form:

$$\begin{aligned} J(\theta) &= \frac{1}{2} \sum_{i=1}^n (\mathbf{x}_i^T \theta - y_i)^2 \\ &= \frac{1}{2} (X\theta - \bar{y})^T (X\theta - \bar{y}) \\ &= \frac{1}{2} (\theta^T X^T X \theta - \theta^T X^T \bar{y} - \bar{y}^T X \theta + \bar{y}^T \bar{y}) \end{aligned}$$

To minimize $J(\theta)$, take derivative (gradient) and set to zero: $\nabla_{\theta} J(\theta) = 0$

$$\Rightarrow X^T X \theta = X^T \bar{y}$$

The normal equations

$$\Downarrow \\ \theta^* = (X^T X)^{-1} X^T \bar{y}$$

Closed form
solution

See handout 4.1 + 4.3 $\Rightarrow$ matrix calculus, partial deri $\Rightarrow$ Gradient

$$\nabla_{\theta} (\theta^T \bar{X}^T \bar{X} \theta) = 2 \bar{X}^T \bar{X} \theta \quad (P24)$$

$$\nabla_{\theta} (-2 \theta^T \bar{X}^T y) = -2 \bar{X}^T y \quad (P24)$$

$$\nabla_{\theta} (y^T y) = 0$$

$$\Rightarrow \nabla_{\theta} J(\theta) = \boxed{\bar{X}^T \bar{X} \theta - \bar{X}^T y}$$

Detailed
Derivation is
Optional /
Extra

$$\Rightarrow \text{Hessian } H(J(\theta)) = \frac{\partial \nabla_{\theta} J(\theta)}{\partial \theta} = \frac{\partial (\bar{X}^T \bar{X} \theta)}{\partial \theta} =$$

Gram matrix $\bar{X}^T \bar{X}$

Advanced / Optional / More in
Extra Slides

Method I: normal equations to minimize the loss

- Write the cost function in matrix form:

$$J(\theta) = \frac{1}{2} \sum_{i=1}^n (\mathbf{x}_i^T \theta - y_i)^2$$

$$= \frac{1}{2} (X\theta - \bar{y})^T (X\theta - \bar{y})$$

$$= \frac{1}{2} (\underbrace{\theta^T X^T X \theta}_{2 \times n \times n \times 1} - \underbrace{\theta^T X^T \bar{y}}_{1 \times n \times n \times 1} - \underbrace{\bar{y}^T X \theta}_{1 \times n \times n \times 1} + \bar{y}^T \bar{y})$$

$$\nabla_{\theta} J(\theta) = \frac{\partial J(\theta)}{\partial \theta} = \frac{1}{2} (2X^T X \theta - X^T \bar{y} - X^T \bar{y})$$

To minimize $J(\theta)$, take derivative and set to zero:

$$\nabla_{\theta} J(\theta) = 0 \Rightarrow$$

$$X^T X \theta = X^T \bar{y}$$

The normal equations

$$\theta^* = (X^T X)^{-1} X^T \bar{y}$$

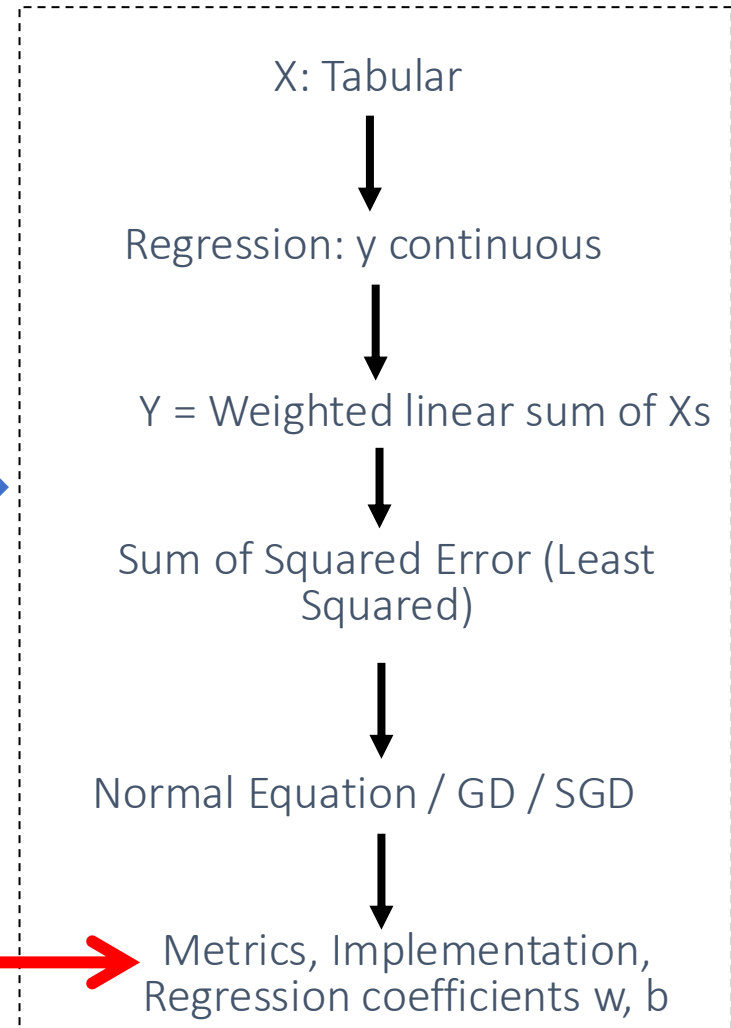
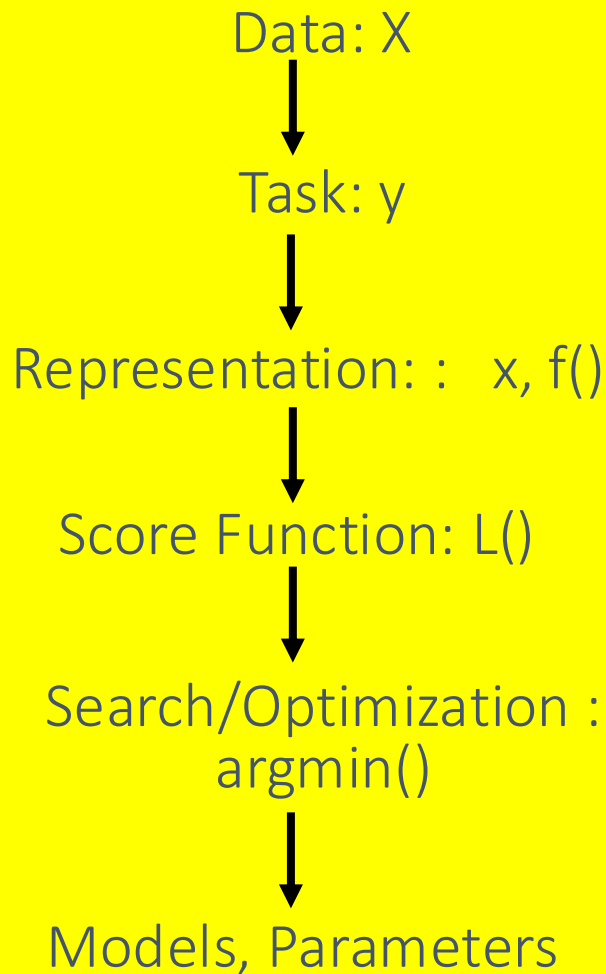
Closed form
solution

Detailed
Derivation is
Optional /
Extra

Thank You



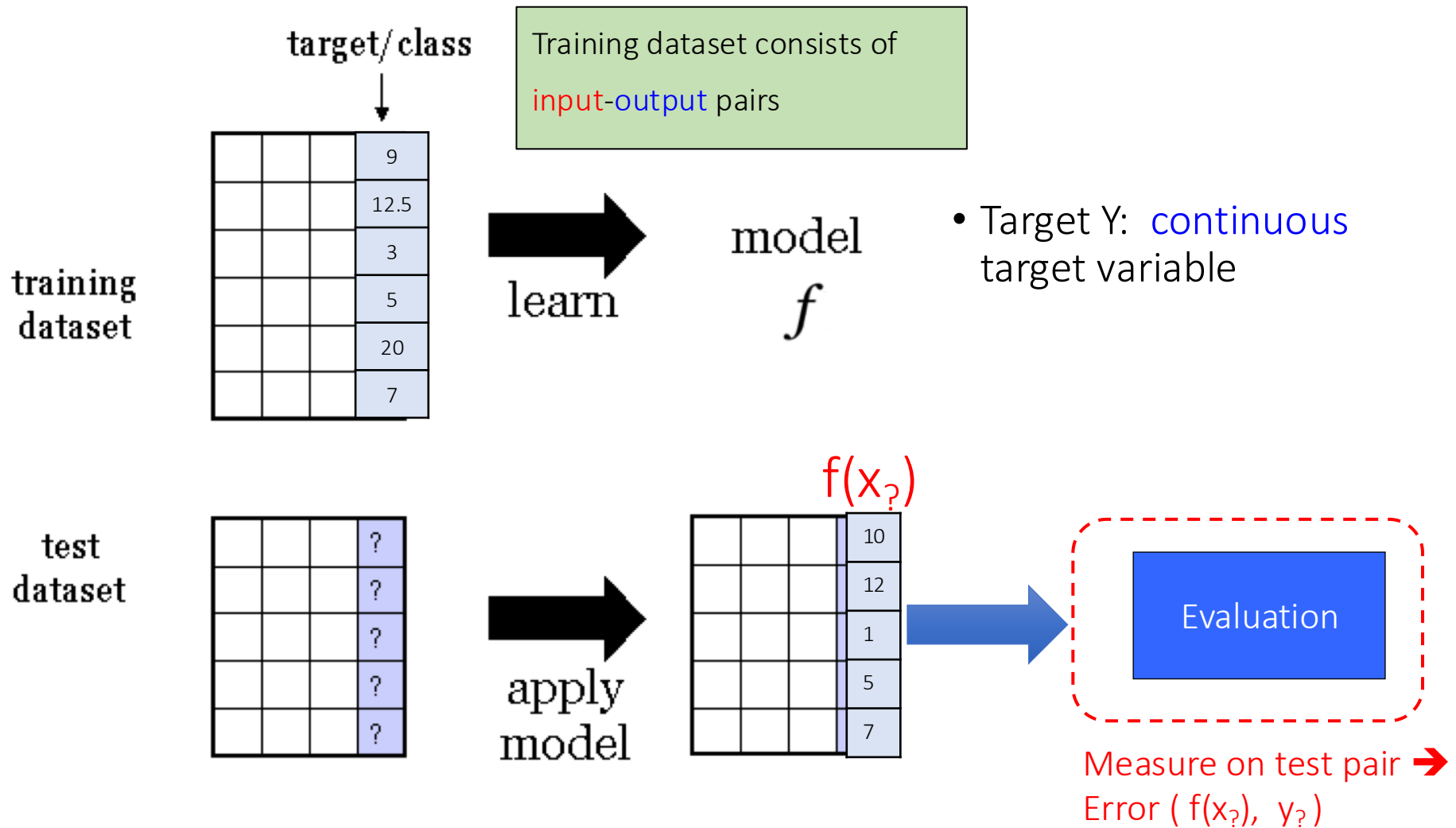
Today: Multivariate Linear Regression in a Nutshell



We aim to make the learned model

- 
- 1. Generalize Well
 - 2. Computational Scalable and Efficient
 - 3. Robust / Trustworthy / Interpretable
 - Especially for some domains, it is about trust!

How to know the regression program works well: Metrics on Regression Predictions



```
# Train the model using the training sets
regr.fit(diabetes_X_train, diabetes_y_train)

# Make predictions using the testing set
diabetes_y_pred = regr.predict(diabetes_X_test)
```

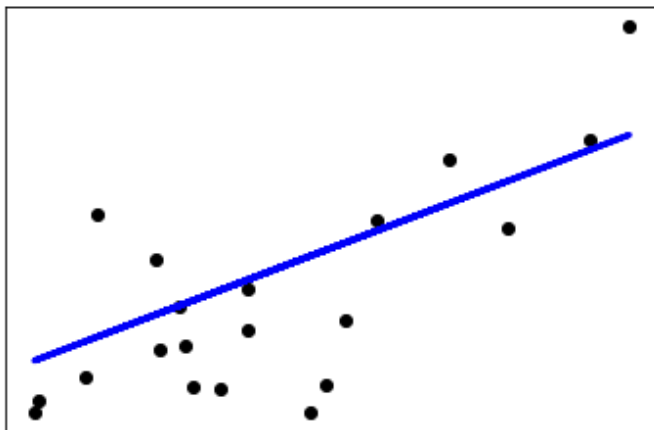
```
# The coefficients
print('Coefficients: \n', regr.coef_)
# The mean squared error
print('Mean squared error: %.2f'
      % mean_squared_error(diabetes_y_test, diabetes_y_pred))
# The coefficient of determination: 1 is perfect prediction
print('Coefficient of determination: %.2f'
      % r2_score(diabetes_y_test, diabetes_y_pred))
```

Mean squared error: 2548.07
Coefficient of determination: 0.47

```
# Plot outputs
plt.scatter(diabetes_X_test, diabetes_y_test, color='black')
plt.plot(diabetes_X_test, diabetes_y_pred, color='blue', linewidth=3)

plt.xticks(())
plt.yticks(())

plt.show()
```



```
# Train the model using the training sets
regr.fit(diabetes_X_train, diabetes_y_train)

# Make predictions using the testing set
diabetes_y_pred = regr.predict(diabetes_X_test)
```

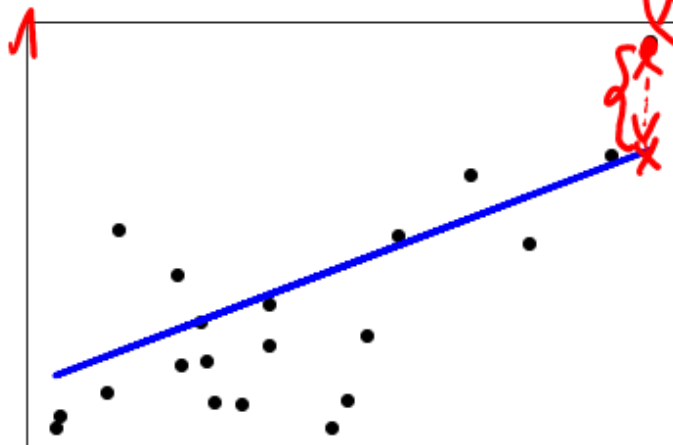
```
# The coefficients
print('Coefficients: \n', regr.coef_)
# The mean squared error
print('Mean squared error: %.2f'
      % mean_squared_error(diabetes_y_test, diabetes_y_pred))
# The coefficient of determination: 1 is perfect prediction
print('Coefficient of determination: %.2f'
      % r2_score(diabetes_y_test, diabetes_y_pred))
```

Mean squared error: 2548.07
Coefficient of determination: 0.47

```
# Plot outputs
plt.scatter(diabetes_X_test, diabetes_y_test, color='black')
plt.plot(diabetes_X_test, diabetes_y_pred, color='blue', linewidth=3)

plt.xticks(())
plt.yticks(())

plt.show()
```



$$(x_{t1}, y_{t1}) \dots (x_{ts}, y_{ts})$$

$$\hat{y}_{t1} \dots \hat{y}_{ts}$$

$$\frac{1}{ts} \sum_{i=1}^{ts} |y_{ti} - \hat{y}_{ti}|$$

mean-absolute
-error

- Test MSE Error to report:

$$J_{test_MSE} = \frac{1}{m} \sum_{i=n+1}^{n+m} (\mathbf{x}_i^T \theta^t - y_i)^2$$

$$\begin{array}{l} \text{training} \\ \text{dataset} \end{array} \mathbf{X}_{train} = \begin{bmatrix} - & - & \mathbf{x}_1^T & - & - \\ - & - & \mathbf{x}_2^T & - & - \\ \vdots & & \vdots & & \vdots \\ - & - & \mathbf{x}_n^T & - & - \end{bmatrix} \quad \bar{\mathbf{y}}_{train} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

$$\begin{array}{l} \text{test} \\ \text{dataset} \end{array} \mathbf{X}_{test} = \begin{bmatrix} - & - & \mathbf{x}_{n+1}^T & - & - \\ - & - & \mathbf{x}_{n+2}^T & - & - \\ \vdots & & \vdots & & \vdots \\ - & - & \mathbf{x}_{n+m}^T & - & - \end{bmatrix} \quad \bar{\mathbf{y}}_{test} = \begin{bmatrix} y_{n+1} \\ y_{n+2} \\ \vdots \\ y_{n+m} \end{bmatrix}$$

Many other possible Metrics for Regression

https://scikit-learn.org/stable/modules/model_evaluation.html

3.3. Metrics and scoring: quantifying the quality of predictions

3.3.1. The **scoring** parameter:

Defining model evaluation rules

3.3.2. Classification metrics

3.3.3. Multilabel ranking metrics

3.3.4. Regression metrics

3.3.5. Clustering metrics

3.3.6. Dummy estimators

Regression

'explained_variance'	<code>metrics.explained_variance</code>
' <u>max_error</u> '	<code>metrics.max_error</code>
'neg_mean_absolute_error'	<code>metrics.mean_absolute_error</code>
'neg_mean_squared_error'	<code>metrics.mean_squared_error</code>
'neg_root_mean_squared_error'	<code>metrics.mean_squared_error</code>
'neg_mean_squared_log_error'	<code>metrics.mean_squared_log_error</code>
'neg_median_absolute_error'	<code>metrics.median_absolute_error</code>
' <u>r2</u> '	<code>metrics.r2_score</code>
'neg_mean_poisson_deviance'	<code>metrics.mean_poisson_deviance</code>
'neg_mean_gamma_deviance'	<code>metrics.mean_gamma_deviance</code>

Usage examples:

Question 3.1. Linear Regression+ Train-Test Split

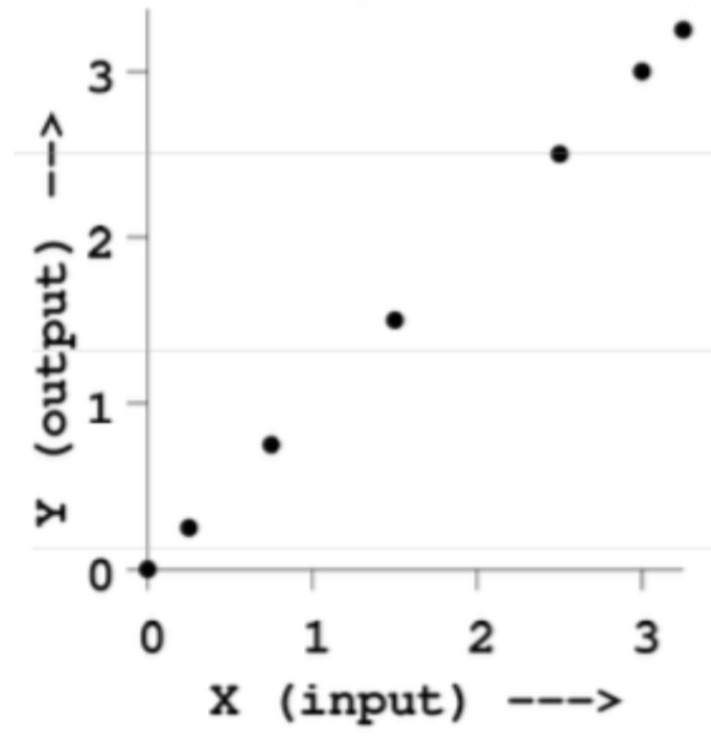


Figure 1: A reference dataset for regression with one real-valued input (x as horizontal axis) and one real-valued output (y as vertical axis).

What is the mean squared training error when running linear regression to fit the data ? (i.e., the model is $y = \beta_0 + \beta_1 x$). Assuming the rightmost three points are in the test set, and the others are in the training set. (you can eyeball the answers.)

We aim to make the learned model

- 1. Generalize Well



- 2. Computational Scalable and Efficient

- 3. Robust / Trustworthy / Interpretable
 - Especially for some domains, this is about trust!

Why we Prefer Concise Vector/Matrix Form?

Training: Closed
form solution

$$\overset{\text{fit}}{\theta^*} = (X_{train}^T X_{train})^{-1} (X_{train}^T \vec{y}_{train})$$

Testing: on
multiple Test
Inputs

$$\hat{\vec{y}}_{test} \overset{\text{predict}}{=} X_{test} \theta^*$$

Extra: Naïve Matrix Multiply

$n \times n$

$\{$

$\{ \text{implements } C = C + A * B \}$

for i = 1 to n

for j = 1 to n

$o(n)$

$[$

for k = 1 to n

$C(i,j) = C(i,j) + A(i,k) * B(k,j)$

$]$

$\}$

A, B, C
 $n \times n \quad n \times n \quad n \times n$

$\}$

$O(n^3)$
time

Algorithm has $2 * n^3 = O(n^3)$ Flops and operates on $3 * n^2$ words of memory

$\underbrace{\hspace{10em}}_{\text{space cost}}$

$\{$

$A(i,:)$

$B(:,j)$

$\}$

59

http://www.cs.berkeley.edu/~demmelm/cs267_Spr12/

Computational Cost (Naïve Way)

$$\mathbf{X}_{\text{train}} \rightarrow \mathbf{X}^T$$

$n \times p$ $p \times n$

$$\vec{\theta}^* = \left(\underbrace{\mathbf{X}^T \mathbf{X}}_{\substack{p \times n \quad n \times p \\ p \times p}} \right)^{-1} \mathbf{X}^T \vec{y}$$

$$\mathbf{X}^T \mathbf{X} : O(p^2 n)$$

$$\left[(\mathbf{X}^T \mathbf{X})^{-1} : O(p^3) \right] O(\underline{np^2} + \underline{p^3})$$

when $n \gg p$, matrix multi.
slower than inversion

Extra:

Many architecture details and Algorithm details to consider

- (1): Data parallelization through CPU SIMD / Multithreading/ GPU parallelization /
- (2): Memory hierarchical / locality
- (3): Better algorithms, like Strassen's Matrix Multiply and many others

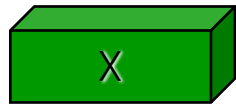
(1): SIMD: Single Instruction, Multiple Data

- Scalar processing

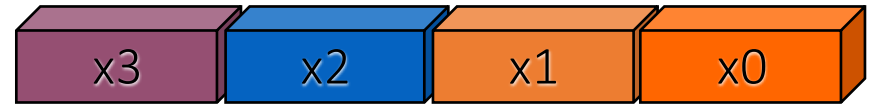
- traditional mode
- one operation produces one result

$$= \sum_{i=0}^3 (x_i + y_i)$$

x_3, x_2, x_1, x_0



X



- SIMD processing

- with SSE / SSE2
- SSE = streaming SIMD extensions
- one operation produces multiple results

Extra:

+

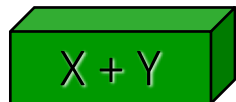


y_3, y_2, y_1, y_0

Y



+



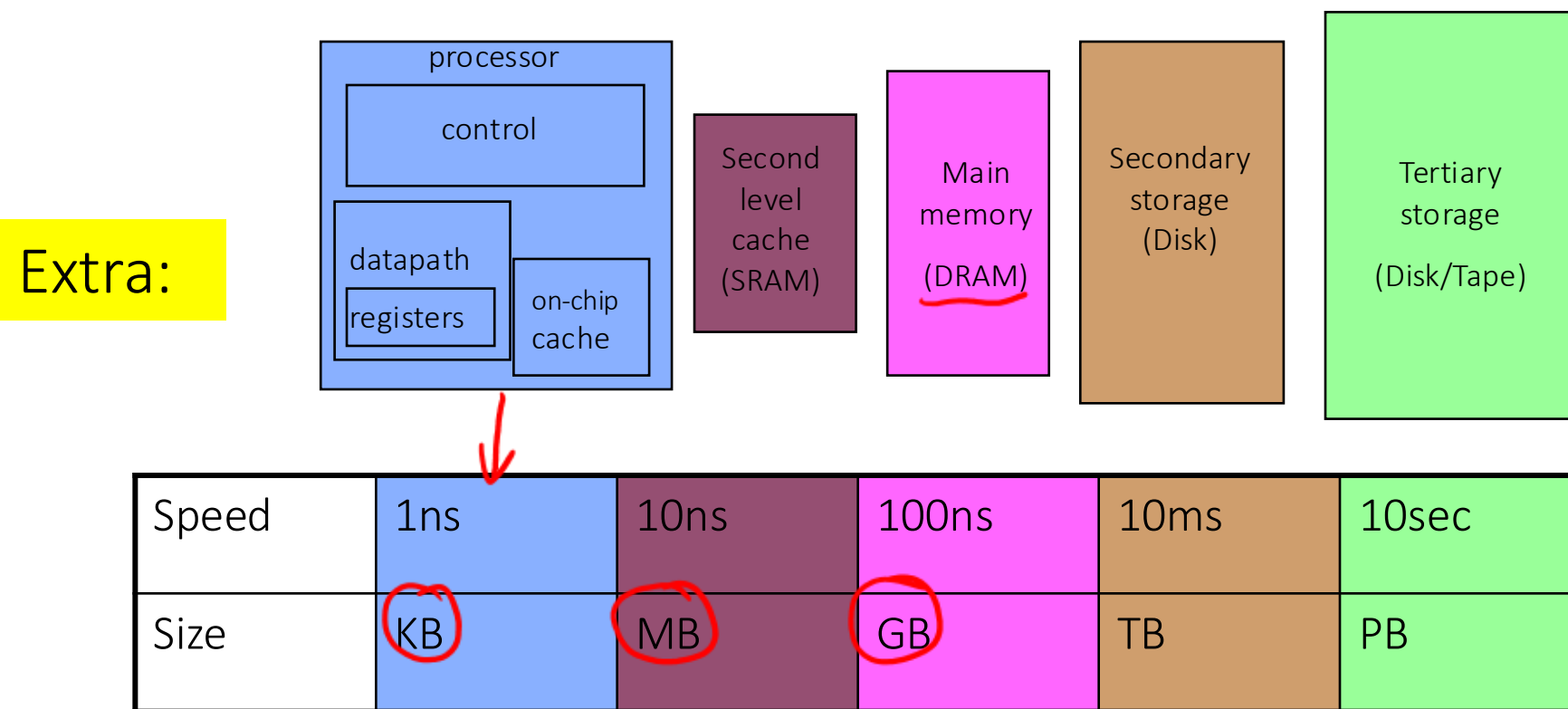
X + Y



Slide Source: Alex Klimovitski & Dean Macri, Intel Corporation

(2): Memory Hierarchy

- Most programs have a high degree of **locality** in their accesses
 - **spatial locality**: accessing things nearby previous accesses
 - **temporal locality**: reusing an item that was previously accessed
- Memory hierarchy tries to exploit locality to improve average



The following complexity figures assume that arithmetic with individual elements has complexity $O(1)$, as is the case with fixed-precision operations on a [finite field](#).

(3)

Operation	Input	Output	Algorithm	Complexity
Matrix multiplication	Two $n \times n$ matrices	One $n \times n$ matrix	Schoolbook matrix multiplication	$O(n^3)$
			Strassen algorithm	$O(n^{2.807})$
			Coppersmith–Winograd algorithm	$O(n^{2.376})$
			Optimized CW-like algorithms ^{[14][15][16]}	$O(n^{2.373})$
Matrix multiplication	One $n \times m$ matrix & one $m \times p$ matrix	One $n \times p$ matrix	Schoolbook matrix multiplication	$O(nmp)$
Matrix inversion*	One $n \times n$ matrix	One $n \times n$ matrix	Gauss–Jordan elimination	$O(n^3)$
			Strassen algorithm	$O(n^{2.807})$
			Coppersmith–Winograd algorithm	$O(n^{2.376})$
			Optimized CW-like algorithms	$O(n^{2.373})$
Singular value decomposition	One $m \times n$ matrix	One $m \times m$ matrix, one $m \times n$ matrix, & one $n \times n$ matrix		$O(mn^2)$ ($m \leq n$)
		One $m \times r$ matrix, one $r \times r$ matrix, & one $n \times r$ matrix		
Determinant	One $n \times n$ matrix	One number	Laplace expansion	$O(n!)$
			Division-free algorithm ^[17]	$O(n^4)$
			LU decomposition	$O(n^3)$
			Bareiss algorithm	$O(n^3)$
			Fast matrix multiplication ^[18]	$O(n^{2.373})$
Back substitution	Triangular matrix	n solutions	Back substitution ^[19]	$O(n^2)$

$$\partial J(w, b) = 0$$

Extra:

Basic Linear Algebra Subroutines (BLAS) → numpy: a wrapper library of BLAS

- Industry standard interface (evolving)
 - www.netlib.org/blas, www.netlib.org/blas/blast-forum
- Vendors, others supply optimized implementations
- History
 - BLAS1 (1970s):
 - vector operations: dot product, saxpy ($y = a * x + y$), etc
 - $m = 2 * n$, $f = 2 * n$, $q = f / m = \text{computational intensity} \sim 1$ or less
 - BLAS2 (mid 1980s)
 - matrix-vector operations: matrix vector multiply, etc
 - $m = n^2$, $f = 2 * n^2$, $q \sim 2$, less overhead
 - somewhat faster than BLAS1
 - BLAS3 (late 1980s)
 - matrix-matrix operations: matrix matrix multiply, etc
 - $m \leq 3n^2$, $f = O(n^3)$, so $q = f / m$ can possibly be as large as n , so BLAS3 is potentially much faster than BLAS2
- Good algorithms use BLAS3 when possible (LAPACK & ScaLAPACK)
 - See www.netlib.org/{lapack,scalapack}

Functionality [\[edit \]](#)

BLAS functionality is categorized into three sets of routines called "levels", which correspond to both the chronological order of definition and publication, as well as the degree of the polynomial in the complexities of algorithms; Level 1 BLAS operations typically take **linear time**, $O(n)$, Level 2 operations quadratic time and Level 3 operations cubic time.^[18] Modern BLAS implementations typically provide all three levels.

Level 1 [\[edit \]](#)

This level consists of all the routines described in the original presentation of BLAS (1979),^[1] which defined only *vector operations* on **strided arrays**: **dot products**, **vector norms**, a generalized vector addition of the form

$$y \leftarrow \alpha x + y$$

(called "axpy") and several other operations.

Level 2 [\[edit \]](#)

This level contains *matrix-vector operations* including, among other things, matrix-vector multiplication (gemv):

$$y \leftarrow \alpha Ax + \beta y$$

as well as a solver for x in the linear equation

$$Tx = y$$

with T being triangular. Design of the Level 2 BLAS started in 1984, with re in 1988.^[19] The Level 2 subroutines are especially intended to improve per programs using BLAS on **vector processors**, where Level 1 BLAS are subo "because they hide the matrix-vector nature of the operations from the com

Level 3 [\[edit \]](#)

This level, formally published in 1990,^[18] contains *matrix-matrix operations* "general **matrix multiplication**" (**gemm**), of the form

$$C \leftarrow \alpha AB + \beta C$$

where A and B can optionally be **transposed** or **hermitian-conjugated** inside and all three matrices may be strided. The ordinary matrix multiplication $A \cdot B$ performed by setting α to one and C to an all-zeros matrix of the appropriat

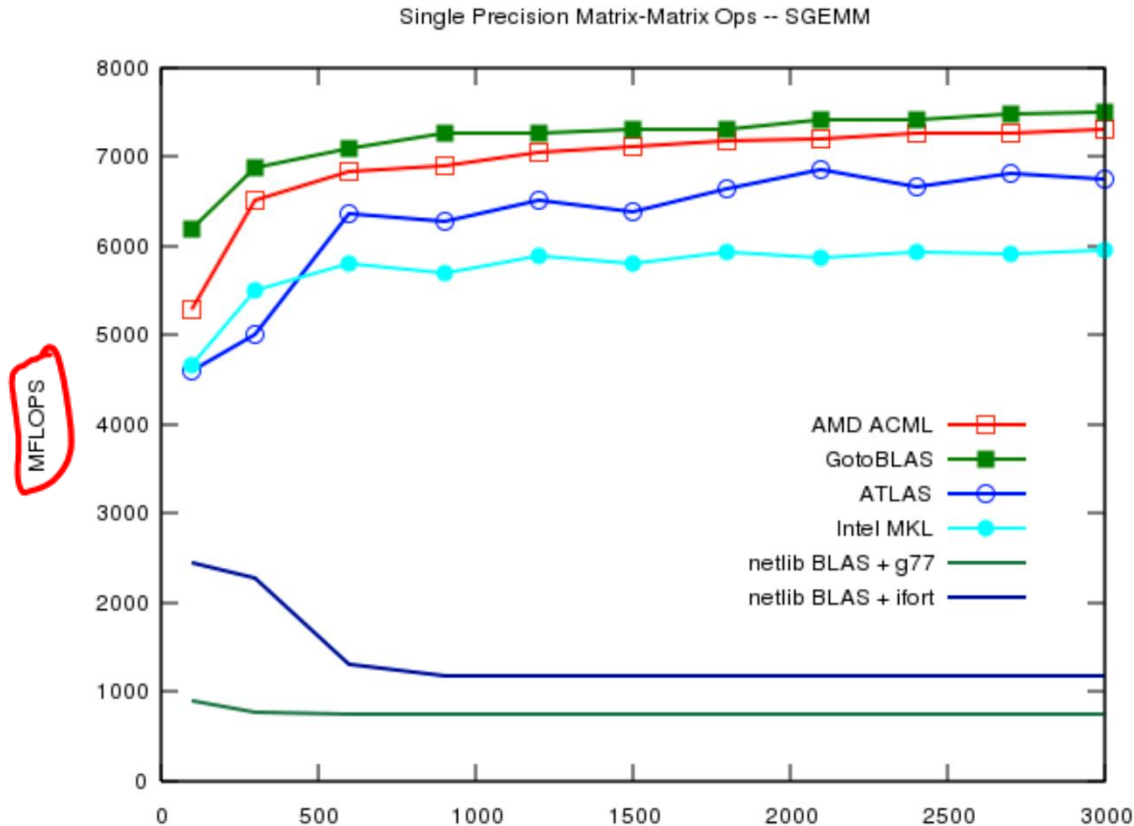
Also included in Level 3 are routines for solving

$$B \leftarrow \alpha T^{-1} B$$

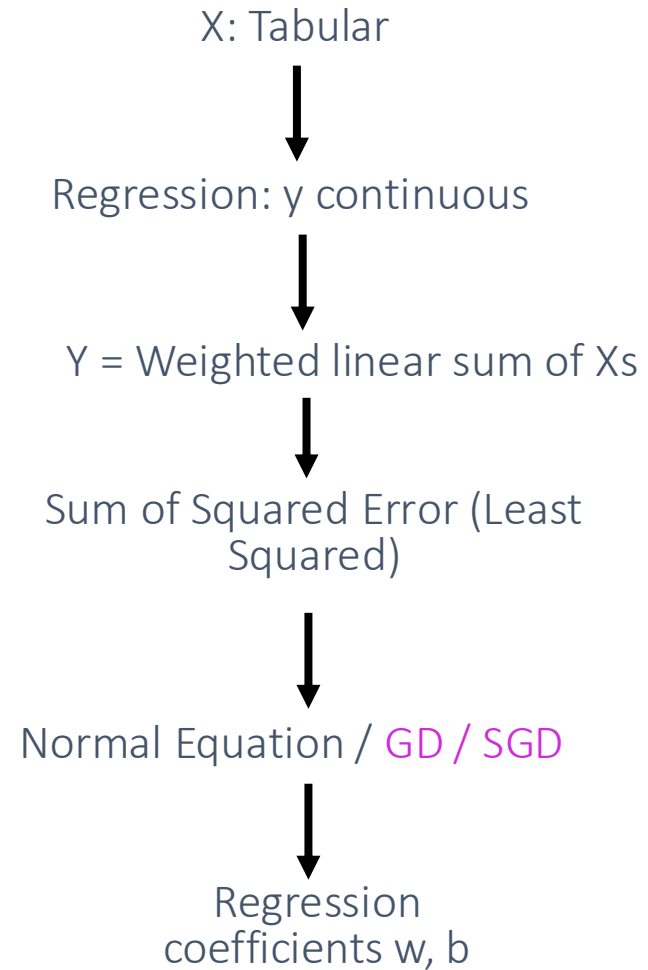
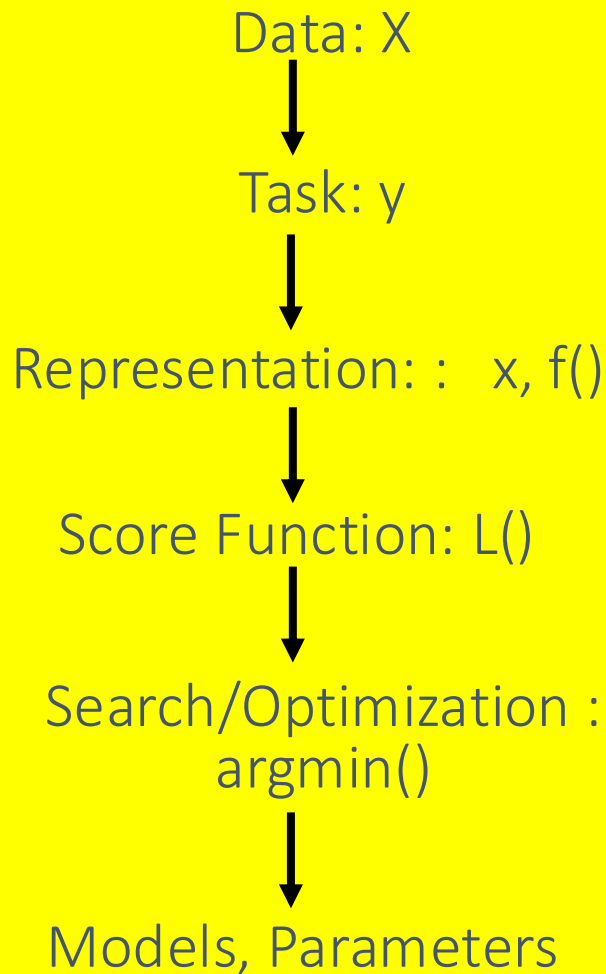
where T is a triangular matrix, among other functionality.

BLAS performance is very much system dependent, e.g., https://www.hoffman2.idre.ucla.edu/blas_benchmark/

Versions of BLAS compared: BLAS library from the Netlib Repository, ATLAS library, Intel-MKL library, AMD ACML Library and Goto BLAS.



Recap : Multivariate Linear Regression in a Nutshell



Thank You



References

- Big thanks to Prof. Eric Xing @ CMU for allowing me to reuse some of his slides

□ <http://www.cs.cmu.edu/~zkolter/course/15-884/linalg-review.pdf>

□ Plus S1-L3-extra.pdf in course schedule page

Review (I):

- Sum the Squared Elements of a Vector equals Vector dot product to itself

$$\vec{a} = \sum \theta - y$$

$$\mathbf{a} = \begin{bmatrix} 5 \\ 2 \\ 8 \end{bmatrix}$$

$$J(\theta) = \frac{1}{2} \sum_{i=1}^n (\mathbf{x}_i^T \theta - y_i)^2$$

$$\mathbf{a}^T = \begin{bmatrix} \square & \square \end{bmatrix} \begin{bmatrix} 5 & 2 & 8 \end{bmatrix}$$

$$\vec{a}^T \vec{a} = \sum_{i=1}^n a_i^2$$

$$\mathbf{a}^T \mathbf{a} = \begin{bmatrix} 5 & 2 & 8 \end{bmatrix} \begin{bmatrix} 5 \\ 2 \\ 8 \end{bmatrix} = 5^2 + 2^2 + 8^2 = 93$$

Review(I) :

$$\mathbf{a} = \begin{bmatrix} \mathbf{x}_1^T \boldsymbol{\theta} - y_1 \\ \mathbf{x}_2^T \boldsymbol{\theta} - y_2 \\ \vdots \\ \mathbf{x}_n^T \boldsymbol{\theta} - y_n \end{bmatrix} = X\boldsymbol{\theta} - \vec{y}$$

$$\mathbf{a}^T \mathbf{a} = 2J(\boldsymbol{\theta}) = \sum_{i=1}^n (\mathbf{x}_i^T \boldsymbol{\theta} - y_i)^2$$

$$\begin{aligned}
J(\theta) &= (\mathbf{X}\theta - \mathbf{y})^T (\mathbf{X}\theta - \mathbf{y}) \frac{1}{2} \\
&= ((\mathbf{X}\theta)^T - \mathbf{y}^T) (\mathbf{X}\theta - \mathbf{y}) \frac{1}{2} \\
&= (\theta^T \mathbf{X}^T - \mathbf{y}^T) (\mathbf{X}\theta - \mathbf{y}) \frac{1}{2} \\
&= \left(\theta^T \mathbf{X}^T \mathbf{X} \theta - \underbrace{\theta^T \mathbf{X}^T \mathbf{y} - \mathbf{y}^T \mathbf{X} \theta}_{\text{since } \theta^T \mathbf{X}^T \mathbf{y} = \mathbf{y}^T \mathbf{X} \theta} + \mathbf{y}^T \mathbf{y} \right) \frac{1}{2}
\end{aligned}$$

$$\begin{aligned}
&\text{since } \theta^T \mathbf{X}^T \mathbf{y} = \mathbf{y}^T \mathbf{X} \theta \\
&\langle \mathbf{X}\theta, \mathbf{y} \rangle \quad \langle \mathbf{y}, \mathbf{X}\theta \rangle
\end{aligned}$$

$$= \left(\theta^T \mathbf{X}^T \mathbf{X} \theta - 2 \theta^T \mathbf{X}^T \mathbf{y} + \mathbf{y}^T \mathbf{y} \right) \frac{1}{2}$$

$\Rightarrow J(\theta)$ quadratic func of θ ;



Review (II): gradient of linear form

$$\frac{\partial (\theta^T X^T y)}{\partial \theta} = X^T y$$

One
Concrete
Example

$$f(w) = w^T a = [w_1, w_2, w_3] \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} = w_1 + 2w_2 + 3w_3$$

$$\frac{\partial f}{\partial w_1} = 1$$

$$\frac{\partial f}{\partial w_2} = 2$$

$$\frac{\partial f}{\partial w_3} = 3$$

$$\frac{\partial f}{\partial w} = \frac{\partial w^T a}{\partial w} = a = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$

Review (III): Gradient of Quadratic Form

- See S1-L3-extra.pdf -> Page 17, Page 23-24
- See white board

$$\frac{\partial (\theta^T X^T X \theta)}{\partial \theta} = \frac{\partial (\theta^T G \theta)}{\partial \theta} = 2G\theta = 2X^T X \theta$$

Review (III): Single Var-Func to Multivariate

Single Var-Function	Multivariate Calculus
Derivative	Partial Derivative
Second-order derivative	Gradient
	Directional Partial Derivative
	Vector Field
	Contour map of a function
	Surface map of a function
	Hessian matrix
	Jacobian matrix (vector in / vector out)

Review (IV) : Definitions of gradient (Matrix_calculus / Scalar-by-vector)

- Size of gradient is always the same as the size of variable

$$\nabla_x f(x) = \begin{bmatrix} \frac{\partial f(x)}{\partial x_1} \\ \frac{\partial f(x)}{\partial x_2} \\ \vdots \\ \frac{\partial f(x)}{\partial x_n} \end{bmatrix} \in \mathbb{R}^n \quad \text{if } x \in \mathbb{R}^n$$

In principle, gradients are a natural extension of partial derivatives to functions of multiple variables.

Review (V): Rank of a Matrix

- $\text{rank}(A)$ (the rank of a m -by- n matrix A) is
 - = The maximal number of linearly independent columns
 - = The maximal number of linearly independent rows

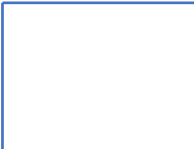
- If A is n by m , then
 - $\text{rank}(A) \leq \min(m, n)$
 - If $n = \text{rank}(A)$, then A has full row rank
 - If $m = \text{rank}(A)$, then A has full column rank

$$\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \quad \begin{pmatrix} 2 & 1 \\ 4 & 2 \end{pmatrix}$$

Rank=? Rank=?

If A is $n \times n$, $\text{rank}(A) = n$ iff A is invertible

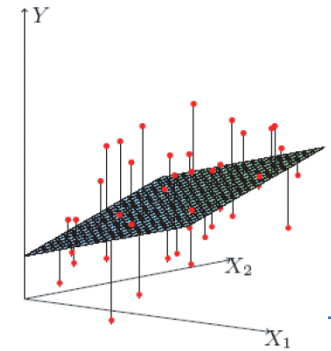
$\text{rank}(AB) \leq \min(\text{rank}(A), \text{rank}(B))$



EXTRA

In Case you are interested in more advanced details!

Probabilistic Interpretation of Linear Regression (Extra)



- Let us assume that the target variable and the inputs are related by the equation:

$$y_i = \theta^T \mathbf{x}_i + \varepsilon_i$$

error data on each point

where ε is an error term of unmodeled effects or random noise

- Now assume that ε follows a Gaussian $N(0, \sigma)$, then we have:

$$p(y_i | x_i; \theta) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y_i - \theta^T \mathbf{x}_i)^2}{2\sigma^2}\right)$$

- By iid (among samples) assumption:

$$L(\theta) = \prod_{i=1}^n p(y_i | x_i; \theta) = \left(\frac{1}{\sqrt{2\pi}\sigma}\right)^n \exp\left(-\frac{\sum_{i=1}^n (y_i - \theta^T \mathbf{x}_i)^2}{2\sigma^2}\right)$$

Many more variations of LinearR from this perspective, e.g. binomial / poisson (LATER)

Extra: Loss $J()$ is Convex

$$J(\theta) = \frac{1}{2} (\theta^T X^T X \theta - 2 \theta^T X^T y + y^T y)$$

$$\text{Hessian}(J(\theta)) = X^T X$$

- *This is called Gram matrix that is positive semi-definite (PSD)*

→ $J(\theta)$ is convex

- Convexity means: If $\nabla J(\theta^*) = 0$, then $J(\theta)$ is minimized at θ^*

Review: Hessian Matrix

Derivatives and Second Derivatives

Cost function

$$J(\boldsymbol{\theta})$$

Gradient

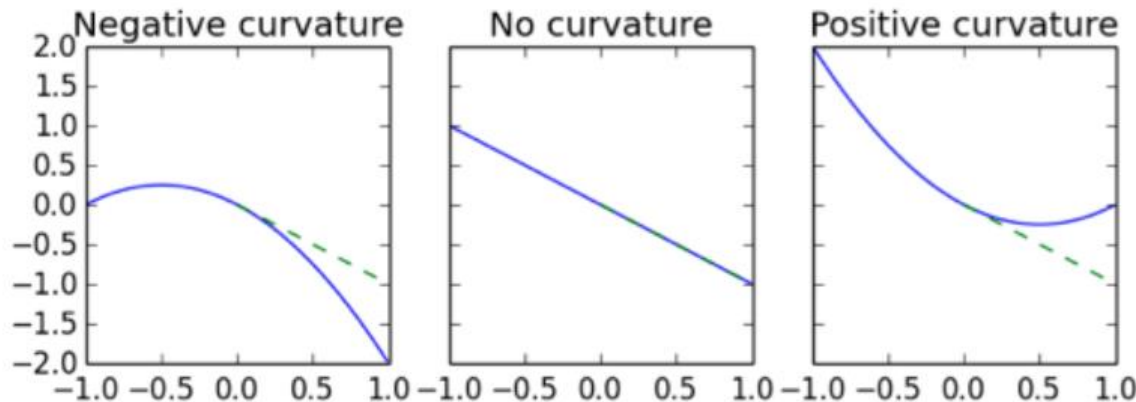
$$\mathbf{g} = \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

$$g_i = \frac{\partial}{\partial \theta_i} J(\boldsymbol{\theta})$$

Hessian

$$\mathbf{H}$$

$$H_{i,j} = \frac{\partial}{\partial \theta_j} g_i$$



H PD
for positive
curvature

Positive Definite Hessian

Extra: Convex function

- Intuitively, a convex function (1D case) has a single point at which the derivative goes to zero, and this point is a minimum.
- Intuitively, a function f (1D case) is convex on the range $[a,b]$ if a function's second derivative is positive every-where in that range.
- Intuitively, if a multivariate function's Hessians is pd (positive definite!), this (multivariate) function is Convex
 - Intuitively, we can think “Positive definite” matrices as analogy to positive numbers in matrix case

Our loss function $J()$'s Hessian is
Positive Semi-definite - PSD

$$H = \mathbf{X}^T \mathbf{X}$$

is always
PSD

Intuitively, a convex function with PD hessian is minimized @ point whose

- ✓ derivative (slope) is zero
- ✓ gradient is zero vector (multivariate case)

Gram Matrix $H = X^T X$

when X is full rank, H is positive definite!

Extra: Gram Matrix $H = X^T X$
is always positive semi-definite!

Because for any vector a

$$a^T X^T X a = \|Xa\|_2^2 \geq 0$$



Besides, when X is full rank,
 H is Positive Definite (PD) and invertible

Comments on the normal equation

when X full rank $\theta^* = (X^T X)^{-1} X^T \vec{y}$

- In most situations of practical interest, the number of data points n is larger than the dimensionality p of the input space and the matrix X is of full column rank. If this condition holds, then it is easy to verify that $X^T X$ is necessarily invertible.

$$n \gg p$$

- The assumption that $X^T X$ is invertible implies that it is positive definite, thus the critical point (by solving gradient to zero) we have found is a minimum.
- What if X has less than full column rank? $\rightarrow$ regularization (later).

when X not full rank, e.g.

e.g. when $p > n \Rightarrow \text{rank}(X) < p \Rightarrow \text{rank}(\underbrace{X^T X}_{p \times p}) \leq \min\{\text{rank}(X)\} < p \Rightarrow \text{not invertible}$

Extra: positive semi-definite!

$$A \in \mathbb{R}^{n \times n}, \quad \forall x \in \mathbb{R}^n$$

$$\text{If } \underset{|x| \times n}{x}^T \underset{n \times n}{A} \underset{n \times 1}{x} \geq 0$$

$\Rightarrow A$ is positive semi-definite (PSD)

$$\text{If } x^T A x > 0$$

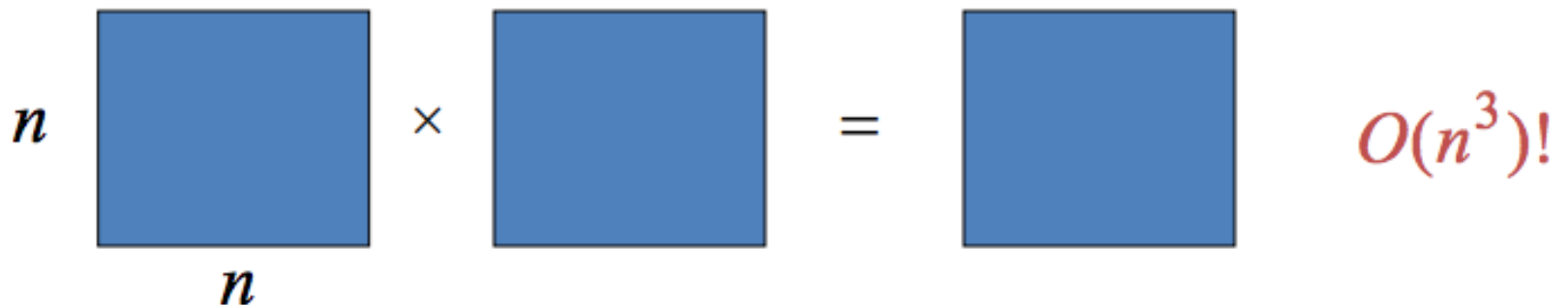
$\Rightarrow A$ is PD $\Rightarrow$ full rank / invertible

See proof on S1-L3-extra: Page 18

Extra: Scalability to big data?

- Traditional CS view: Polynomial time algorithm, Wow!
- Large-scale learning: Sometimes even $O(n)$ is bad! => Many state-of-the-art solutions (e.g., low rank, sparse, hardware, sampling, randomized...)

Simple example: Matrix multiplication

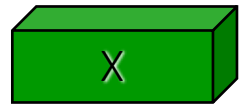


The diagram illustrates matrix multiplication. It shows two blue squares representing $n \times n$ matrices. The first square has a label n to its left and another n below it. This is followed by a multiplication symbol $\times$, then another blue square representing an $n \times n$ matrix. This is followed by an equals sign $=$, then a third blue square representing the resulting $n \times n$ matrix. To the right of the result matrix is the text $O(n^3)!$ in red.

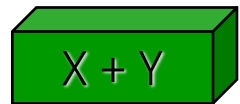
$$\begin{matrix} n \\ \square \\ n \end{matrix} \times \square = \square \quad O(n^3)!$$

SIMD: Single Instruction, Multiple Data

- Scalar processing
 - traditional mode
 - one operation produces one result



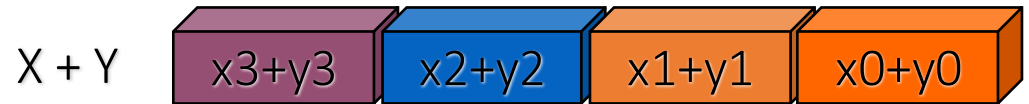
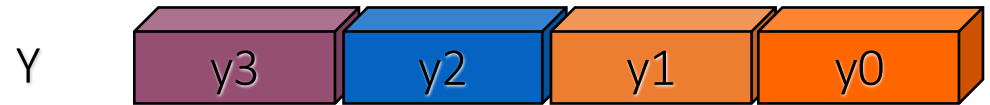
+



- SIMD processing
 - with SSE / SSE2
 - SSE = streaming SIMD extensions
 - one operation produces multiple results



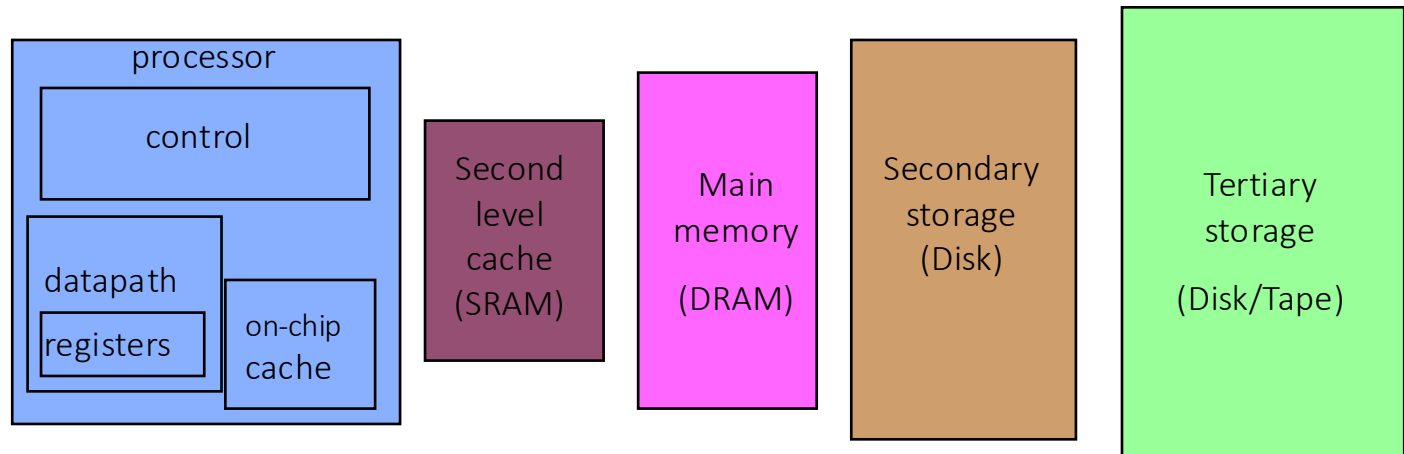
+



Slide Source: Alex Klimovitski & Dean Macri, Intel Corporation

Memory Hierarchy

- Most programs have a high degree of **locality** in their accesses
 - **spatial locality**: accessing things nearby previous accesses
 - **temporal locality**: reusing an item that was previously accessed
- Memory hierarchy tries to exploit locality to improve average

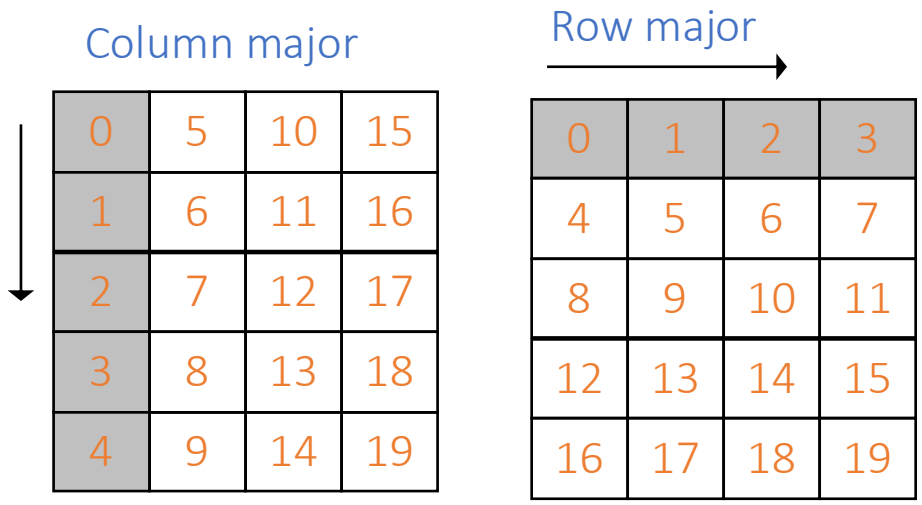


Speed	1ns	10ns	100ns	10ms	10sec
Size	KB	MB	GB	TB	PB

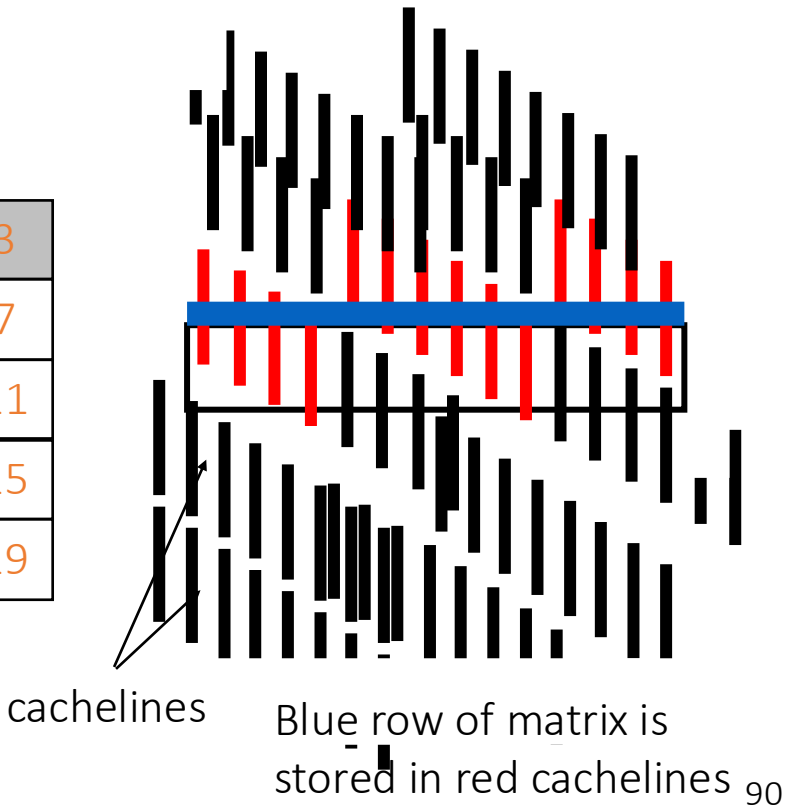
Note on Matrix Storage

- A matrix is a 2-D array of elements, but memory addresses are “1-D”
- Conventions for matrix layout
 - by column, or “column major” (Fortran default); $A(i,j)$ at $A+i*j*n$
 - by row, or “row major” (C default) $A(i,j)$ at $A+i*n+j$
 - recursive (later)

Column major matrix in memory



- Column major (for now)



Strassen's Matrix Multiply

- The traditional algorithm (with or without tiling) has $O(n^3)$ flops
- Strassen discovered an algorithm with asymptotically lower flops
 - $O(n^{2.81})$
- Consider a 2x2 matrix multiply, normally takes 8 multiplies, 4 adds
 - Strassen does it with 7 multiplies and 18 adds

Let $M = \begin{bmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$

Let $p_1 = (a_{12} - a_{22}) * (b_{21} + b_{22})$

$p_5 = a_{11} * (b_{12} - b_{22})$

$p_2 = (a_{11} + a_{22}) * (b_{11} + b_{22})$

$p_6 = a_{22} * (b_{21} - b_{11})$

$p_3 = (a_{11} - a_{21}) * (b_{11} + b_{12})$

$p_7 = (a_{21} + a_{22}) * b_{11}$

$p_4 = (a_{11} + a_{12}) * b_{22}$

Then $m_{11} = p_1 + p_2 - p_4 + p_6$

$m_{12} = p_4 + p_5$

$m_{21} = p_6 + p_7$

$m_{22} = p_2 - p_3 + p_5 - p_7$

Extends to nxn by divide&conquer

Strassen (continued)

$$\begin{aligned} T(n) &= \text{Cost of multiplying } n \times n \text{ matrices} \\ &= 7 * T(n/2) + 18 * (n/2)^2 \\ &= O(n^{\log_2 7}) \\ &= O(n^{2.81}) \end{aligned}$$

- Asymptotically faster
 - Several times faster for large n in practice
 - Cross-over depends on machine
 - “Tuning Strassen's Matrix Multiplication for Memory Efficiency”, M. S. Thottethodi, S. Chatterjee, and A. Lebeck, in Proceedings of Supercomputing '98
- Possible to extend communication lower bound to Strassen
 - #words moved between fast and slow memory $= \Omega(n^{\log_2 7} / M^{(\log_2 7)/2 - 1}) \sim \Omega(n^{2.81} / M^{0.4})$
 - (Ballard, D., Holtz, Schwartz, 2011)
 - Attainable too